LOVELY PROFESSIONAL UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

A CAPSTONE PROJECT REPORT ON

SHOPSPHERE — E-COMMERCE DATABASE AND ANALYTICS SYSTEM USING MONGODB

Submitted in partial fulfillment of the requirements
for the award of the degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By:
RAJAT YADAV
Registration No: 12308041
Programme: B.Tech CSE

Under the Guidance of:
[TO BE FILLED IN AFTER CONFIRMATION]
Faculty / Project Guide
School of CSE, LPU

Lovely Professional University, Phagwara, Punjab (India)
Academic Session: 2026–2027

CERTIFICATE

This is to certify that the project entitled "ShopSphere — E-Commerce Database and Analytics System Using MongoDB", submitted by RAJAT YADAV (Registration Number: 12308041) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Lovely Professional University, Punjab, is a bona fide record of technical work carried out under my supervision and guidance.

The software architecture, database design, indexing strategies, real-time aggregation pipelines, and full-stack implementation embodied in this report have been executed and verified against genuine e-commerce big data. To the best of my knowledge, the matter presented in this report has not been submitted in part or full to any other University or Institution for the award of any degree or diploma.

_____

Project Guide / Supervisor
Name: [TO BE FILLED IN AFTER CONFIRMATION]
Designation: [TO BE FILLED IN]
Department of Computer Science & Engineering
Lovely Professional University

_____

Head of Department
Department of Computer Science & Engineering
School of Computer Science & Engineering
Lovely Professional University

Date:
Place: Lovely Professional University, Phagwara, Punjab

_____

DECLARATION

I hereby declare that the project report entitled "ShopSphere — E-Commerce Database and Analytics System Using MongoDB", submitted to Lovely Professional University, Phagwara, Punjab, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is an authentic record of my own research, system design, and software engineering work carried out during the academic session 2026–2027.

I affirm that the codebase, database schemas, MongoDB aggregation pipelines, query execution plan optimizations, and documentation presented in this report are original. Where contributions, third-party libraries (FastAPI, React, Vite, PyMongo), or external datasets (Brazilian E-Commerce Public Dataset by Olist, licensed via CC BY-NC-SA 4.0) have been utilized, they have been explicitly cited, referenced, and attributed in accordance with international academic integrity and ethical guidelines.

I further declare that this work has not formed the basis for the award of any other degree, diploma, associate-ship, or fellowship at this or any other university or educational institution.

_____

RAJAT YADAV
Registration No: 12308041
Roll No: [TO BE FILLED IN]
B.Tech Computer Science and Engineering
Lovely Professional University

Date:
Place: Lovely Professional University, Punjab

_____

ACKNOWLEDGEMENT

The successful completion of this comprehensive database systems and full-stack software engineering project has been made possible through the guidance, mentorship, and encouragement of several individuals and academic resources at Lovely Professional University.

First and foremost, I express my profound gratitude to my Project Guide, [TO BE FILLED IN AFTER CONFIRMATION], Department of Computer Science and Engineering, for providing constructive feedback, technical guidance, and valuable insights into NoSQL database architecture throughout the design, implementation, and empirical verification phases of ShopSphere.

I extend my sincere appreciation to the Head of Department, School of Computer Science and Engineering, Lovely Professional University, for providing an environment conducive to technical innovation, high-performance computing labs, and rigorous academic standards.

I am equally thankful to the global open-source software community: MongoDB Inc. for engineering Community Server 8.0 with the WiredTiger storage engine; the FastAPI development team led by Sebastián Ramírez for modern asynchronous Python tooling; the React and Vite core teams for ultra-fast frontend developer experiences; and Olist along with Kaggle for making the authentic Brazilian E-Commerce public dataset available for academic study.

Finally, I express my heartfelt indebtedness to my parents, family members, and peers for their continuous moral support, understanding, and encouragement throughout the duration of this engineering endeavour.

RAJAT YADAV
Registration No: 12308041
B.Tech CSE, Lovely Professional University

ABSTRACT

Modern enterprise e-commerce platforms handle massive transaction volumes, highly dynamic product catalogues, and low-latency analytical demands that frequently expose the scalability and architectural limitations of legacy Relational Database Management Systems (RDBMS). Traditional relational models rely on heavy normalization and multi-table SQL JOIN operations that degrade exponentially under high concurrency, while rigid table schemas inhibit rapid catalog schema evolution. This project presents ShopSphere, an enterprise-grade e-commerce database management and real-time business intelligence analytics platform engineered on MongoDB Community Server 8.0, Python FastAPI, and React 18 with TypeScript.

The core technical contribution of ShopSphere centers on an empirical NoSQL document-modelling paradigm that synthesizes embedded sub-document structures with referenced document architectures. Historical order integrity is achieved by denormalizing purchased line-item snapshots and shipping addresses directly inside order documents, guaranteeing single-read retrieval and price immutability without SQL joins. Customer profiles and product catalog entities are linked via BSON ObjectId references to eliminate uncontrolled data duplication and satisfy document size constraints. The platform ingests and normalizes the genuine Brazilian E-Commerce Public Dataset by Olist, transforming relational CSV tables into 99,441 authentic customer documents, 98,666 order documents, 32,951 product records, 97,709 review records, and 3,095 seller entities in a dedicated production database (`shopsphere_olist`).

To enforce database-level integrity, collection-level `$jsonSchema` validators are deployed directly within the MongoDB engine, rejecting malformed writes at the persistence tier. Query performance optimization is empirically verified using live `explain("executionStats")`, demonstrating an index scan (`IXSCAN`) that examines precisely 1 key and 1 document in 0.0 ms compared to an unindexed collection scan (`COLLSCAN`) that sequentially scans all 32,951 documents. Real-time executive dashboards and business intelligence reporting are powered by 8 multi-stage MongoDB aggregation pipelines leveraging `$match`, `$unwind`, `$lookup`, `$group`, `$facet`, and `$dateToString`. Strict environment isolation guarantees that production historical data remains immutable, while interactive demonstrations operate within an isolated demo collection (`shopsphere_demo`) and automated unit/integration tests execute against a dedicated test database (`shopsphere_test`). All 12 automated test suites pass in 0.69 seconds, verifying that ShopSphere delivers an enterprise-ready, academically rigorous, and scalable NoSQL database foundation.

Keywords: Keywords: MongoDB 8.0, NoSQL Document Modeling, Aggregation Framework, B-Tree Query Optimization, $jsonSchema Validation, FastAPI, React TypeScript, Olist Dataset.

TABLE OF CONTENTS

Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

List of Figures viii

List of Tables ix

1. Introduction 1

1.1 Background of E-Commerce Database Management Systems 1

1.2 Evolution from Relational Models to Document-Oriented NoSQL 2

1.3 Why MongoDB Community 8.0 for Modern E-Commerce Platforms 3

1.4 Overview of the ShopSphere Platform & Architectural Vision 4

2. Problem Statement 5

2.1 The Relational Join Bottleneck in Enterprise E-Commerce 5

2.2 Data Redundancy vs Consistency in Historical Order Records 6

2.3 Real-Time Analytical Demands on Operational Databases 6

2.4 Real-World Data Hygiene, Anonymization & Idempotency 7

3. Objectives of the Project 8

3.1 Primary Academic and Architectural Objectives 8

3.2 Specific Technical Deliverables 8

4. Scope of the Project 10

4.1 Functional Modules in Scope 10

4.2 Explicit Boundaries & Limitations 11

5. Literature Review 12

5.1 NoSQL Databases and Document-Store Paradigms 12

5.2 Storage Engine Architectures: MongoDB WiredTiger Engine 13

5.3 Modern Reactive Web Architectures (FastAPI & React) 14

5.4 The Brazilian E-Commerce Dataset by Olist 15

5.5 Comparative Evaluation: RDBMS vs MongoDB Document Model 16

6. System Requirements (Hardware and Software) 17

6.1 Hardware Specifications 17

6.2 Software & Dependency Stack 17

6.3 Network Topology & Environment Configuration 18

7. System Design and Architecture 19

7.1 Three-Tier Client-Server Architecture 19

7.2 Component-Level Architectural Decomposition 20

7.3 Data Ingestion & ETL Processing Workflow 21
7.4 End-to-End API Request & Query Pipeline 22
7.5 Database Separation Architecture 23
8. Database Design **24**
8.1 MongoDB Collection Architecture & Entity Overview 24
8.2 Embedded vs Referenced Data Modeling Strategy 25
8.3 Detailed Collection Schema Specifications 26
8.4 Database-Level $jsonSchema Validation Engine 29
8.5 B-Tree Indexing Strategy 30
9. Implementation and Methodology **32**
9.1 Authentic Dataset Discovery & Preprocessing Engine 32
9.2 UTF-8 BOM Stripping & Multilingual Category Translation 33
9.3 Deterministic Hashing & Idempotent Upsert Strategy 34
9.4 Anonymization Preserving Data Hygiene 35
9.5 Backend Service & Repository Layer Implementation 35
9.6 Frontend Single-Page Application Implementation 36
10. MongoDB Queries and Operations **37**
10.1 Core CRUD Operations 37
10.2 Production Aggregation Pipelines (8 Real Queries) 39
10.3 Query Execution Optimization (explain('executionStats')) 44
10.4 Schema Validation Enforcement & Write Rejection 46
11. Results and Output Screenshots **47**
11.1 Executive Analytics Dashboard Walkthrough 47
11.2 Product Catalogue & Filtering System 48
11.3 Customer CRM & Demographic Distribution 49
11.4 Transactional Order Fulfillment & Immutability Verification 50
11.5 Customer Reviews & Rating Distributions 51
11.6 Inventory & Stock Movement Audit Trail 52
11.7 Business Intelligence & Aggregation Reports 53
11.8 Interactive MongoDB Concepts & Viva Laboratory 54
11.9 Database Topology & Connection Settings 55
11.10 Customer-Facing Storefront 56
11.11 FastAPI Swagger API Documentation 57
12. Testing and Validation **58**
12.1 Automated Testing Strategy & Isolation Philosophy 58
12.2 Test Suite Execution Results (12/12 Passing) 58

12.3 Comprehensive Test Matrix Table 59
12.4 Frontend TypeScript & Vite Production Build Verification 61
12.5 Historical Immutability & Concurrency Safeguards 61
12.6 Idempotency Verification Matrix 62
13. Challenges Faced **63**
13.1 Complexities of Real-World Big Data Ingestion 63
13.2 Balancing Data Integrity with NoSQL Flexibility 63
13.3 Relational-to-Document Paradigm Shift 64
13.4 Transactional Concurrency on Standalone Instances 64
13.5 Query Performance Optimization on 98k+ Orders 65
14. Conclusion **66**
14.1 Summary of Technical Achievements 66
14.2 Pedagogical and Engineering Insights 67
15. Future Scope **68**
15.1 Replica Set Deployment & Multi-Document ACID Transactions 68
15.2 Sharding & Distributed Horizontal Scaling 68
15.3 Vector Search & AI-Powered Product Recommendations 69
15.4 Change Streams & Real-Time Event Architecture 69
15.5 Security Hardening (JWT, RBAC, Field-Level Encryption) 70
16. References / Bibliography **71**
17. Appendix **73**
Appendix A: Complete MongoDB Shell Commands & Verification Scripts 73
Appendix B: Data Dictionary & Field Specifications Table 75
Appendix C: Step-by-Step Project Setup & Execution Guide 77
Appendix D: Top 15 Academic Viva Voce Questions & Model Answers 79

LIST OF FIGURES

Figure Number	Figure Caption / Description	Page
Figure 1	ShopSphere Full-Stack System Architecture Diagram	20
Figure 2	Olist Brazilian E-Commerce Import & Transformation Pipeline Workflow	21
Figure 3	End-to-End API Request, Pydantic Validation & PyMongo Query Flow	22
Figure 4	Multi-Database Namespace Isolation Architecture (Production, Demo, Test)	23
Figure 5	MongoDB Document Model & Schema Relationships (Embedded vs Referenced)	25
Figure 6	Real Application Screenshot — Executive SaaS Analytics Dashboard	47
Figure 7	Real Application Screenshot — Product Catalogue Management System	48
Figure 8	Real Application Screenshot — Customer CRM & Demographic Accounts Ledger	49
Figure 9	Real Application Screenshot — Transactional Order Management & Immutability Badges	50
Figure 10	Real Application Screenshot — Customer Reviews & Rating Distribution Analytics	51
Figure 11	Real Application Screenshot — Real-Time Inventory & Movement Audit Ledger	52
Figure 12	Real Application Screenshot — Business Intelligence & Multi-Dimensional Aggregation Reports	53
Figure 13	Real Application Screenshot — Interactive MongoDB Concepts & Viva Laboratory	54
Figure 14	Real Application Screenshot — Database Engine Settings, Standalone Topology & Document Volume	55
Figure 15	Real Application Screenshot — Live Customer-Facing Shopping Storefront Experience	56
Figure 16	Real Application Screenshot — FastAPI Interactive OpenAPI (Swagger UI) Documentation	57

LIST OF TABLES

Table Number	Table Title / Description	Page
Table 1	Comparative Evaluation: Relational Database (RDBMS) vs MongoDB Document Model	16
Table 2	System Specifications: Hardware Environment (Minimum vs Recommended)	17
Table 3	Complete Software Dependency Stack and Engine Versions	18
Table 4	Verified MongoDB Collection Counts in Primary Production Database (`shopsphere_olist`)	24
Table 5	Olist Relational CSV Source to MongoDB Document Collection Mapping Reference	33
Table 6	Empirical Query Optimization Benchmark: B-Tree IXSCAN vs Unindexed COLLSCAN	45
Table 7	Comprehensive Pytest Automated Test Execution Matrix (12 Verified Unit & Integration Tests)	59
Table 8	Database Namespace Isolation & State Verification Matrix	62
Table 9	Data Dictionary: Complete Entity & Field Specification Matrix	75

1. INTRODUCTION

1.1 Background of E-Commerce Database Management Systems

Electronic commerce (e-commerce) has evolved into one of the most data-intensive domains in contemporary software engineering. Modern digital storefronts, multi-vendor marketplaces, and omnichannel retail platforms operate under demanding workloads characterized by unpredictable traffic spikes, deeply nested product specifications, and high-frequency read operations. In a typical e-commerce session, users perform faceted catalog filtering, dynamic multi-attribute search, real-time stock verification, checkout processing, and review browsing. These interactions demand sub-second latency regardless of concurrency levels.

For decades, Relational Database Management Systems (RDBMS) such as PostgreSQL, MySQL, and Oracle served as the uncontested foundation of commercial retail systems. Relational engines rely on rigorous Boyce-Codd Normal Form (BCNF) normalization, strict tabular schemas, and ACID transactions. However, as catalog diversity expanded to encompass hundreds of heterogeneous product categories—each with distinct attributes ranging from technical apparel sizing to consumer electronics specifications—the rigidity of relational schemas emerged as an operational bottleneck. The necessity of altering schemas across millions of rows to accommodate new product attributes introduces significant operational friction and costly database downtime.

1.2 Evolution from Relational Models to Document-Oriented NoSQL

To resolve the limitations of relational models under internet-scale workloads, the database industry witnessed a paradigm shift toward Not Only SQL (NoSQL) data storage engines. Among the diverse NoSQL paradigms—key-value stores, wide-column families, graph databases, and document stores—document-oriented databases have emerged as the natural fit for e-commerce domain entities.

Document-oriented databases represent data as self-contained, hierarchically structured JSON or Binary JSON (BSON) documents. Unlike tabular rows that strictly segregate related entities across dozens of foreign-key linked tables, document databases offer data locality of reference. An order document, for example, can natively embed its purchased line items, unit prices, discounts, and shipping addresses within a single cohesive unit of storage. This architectural alignment between application-layer domain objects and database persistence layers eliminates the Object-Relational Impedance Mismatch that has historically plagued object-oriented software engineering.

1.3 Why MongoDB Community 8.0 for Modern E-Commerce Platforms

MongoDB Community Server 8.0 represents the industry standard for production document-oriented NoSQL databases, offering an ideal persistence tier for e-commerce platforms due to several fundamental architectural capabilities:

- **WiredTiger Storage Engine:** High-throughput concurrent writes and reads with page-level concurrency, in-memory caching, and transparent block compression (Snappy).

- **Aggregation Framework:** Powerful in-database data processing pipeline providing over 30 stages ($match, $unwind, $group, $lookup, $facet) for real-time analytical reporting without external batch processing.
- **B-Tree Indexing Architecture:** Versatile indexing engine supporting single-field, compound, text, geospatial, and partial filter indexes to optimize diverse search patterns.
- **Native $jsonSchema Validation:** Collection-level schema enforcement using standard JSON Schema definitions, ensuring strict data hygiene without compromising document polymorphism.
- **Enterprise Consistency Guarantees:** Single-document atomicity guarantees, coupled with distributed multi-document ACID transactions across replica sets.

1.4 Overview of the ShopSphere Platform & Architectural Vision

ShopSphere is an enterprise-grade full-stack e-commerce database management and real-time business intelligence platform engineered to demonstrate advanced NoSQL data modeling, performance optimization, and authentic big data ingestion on MongoDB Community Server 8.0. The system pairs a high-performance Python FastAPI backend with a reactive React 18 and TypeScript frontend bundled via Vite.

Rather than relying on toy synthetic datasets, ShopSphere is architected and empirically validated against the authentic Brazilian E-Commerce Public Dataset by Olist, comprising over 100,000 real-world commercial transactions. The platform provides an Executive Dashboard, Product Catalog Management, Customer Relationship Management (CRM), Transactional Order Fulfillment, Inventory Movement Auditing, Customer Sentiment Reviews, and a dedicated MongoDB Concepts Laboratory designed for academic evaluation and technical viva voce demonstrations.

2. PROBLEM STATEMENT

2.1 The Relational Join Bottleneck in Enterprise E-Commerce

In a conventional relational e-commerce schema, fulfilling a standard customer order inquiry requires executing SQL queries spanning at least five to seven normalized tables: `orders`, `order_items`, `products`, `customers`, `addresses`, `payments`, and `discounts`. As database scale expands beyond millions of historical records, relational JOIN operations consume substantial CPU and buffer pool memory. Under high concurrency, locking mechanisms and index traversal across normalized foreign key hierarchies induce significant query latency, degrading the user experience and increasing infrastructure costs.

2.2 Data Redundancy vs Consistency in Historical Order Records

A critical vulnerability in poorly designed relational databases is the temporal mutation of historical transaction records. If an order table merely references a `products` table via foreign key, any subsequent price adjustment or product description modification in the master catalog inadvertently alters the historical record of past orders. Preventing this requires creating synthetic snapshot tables or historical audit ledgers, introducing database complexity. A primary challenge addressed in this project is designing an embedded document architecture that atomically freezes line-item prices and product snapshots at the instant of checkout, guaranteeing historical immutability.

2.3 Real-Time Analytical Demands on Operational Databases

Executive decision-makers require real-time visibility into Gross Revenue, Monthly Sales Trends, Category Performance Breakdown, and Customer Lifetime Value (LTV). Traditionally, organizations deploy complex Extract, Transform, Load (ETL) pipelines to replicate operational data into external Data Warehouses (OLAP), incurring sync delays of hours or days. A central problem addressed by ShopSphere is executing low-latency analytical aggregations directly on the operational MongoDB database without degrading concurrent transaction throughput.

2.4 Real-World Data Hygiene, Anonymization & Idempotency

Working with authentic real-world data—specifically the Brazilian E-Commerce Public Dataset by Olist—presents significant data engineering hurdles. The raw dataset contains non-uniform characters (UTF-8 BOM), multilingual categories (Portuguese requiring English normalization), anonymized customer emails (null values), and repeated review identifiers. Ingesting this data without fabricating synthetic contact details, violating database-level unique constraints, or corrupting document relationships requires an idempotent ETL pipeline with deterministic surrogate keys and partial filter indexing.

3. OBJECTIVES OF THE PROJECT

3.1 Primary Academic and Architectural Objectives

The overarching aim of this project is to research, design, implement, and validate an enterprise-grade e-commerce database management and business intelligence analytics system utilizing modern NoSQL document modeling principles on MongoDB Community Server 8.0.

3.2 Specific Technical Deliverables

To realize the primary aim, the project focuses on seven rigorous technical objectives:

- **1. Advanced Document Modeling:** Formulate a hybrid data model that embeds 1-to-few bounded arrays (purchased items, payment installments, shipping addresses) within orders to ensure historical immutability, while referencing unbounded entities (customers, products) via ObjectIds to eliminate duplication.
- **2. High-Performance CRUD Primitives:** Implement complete Create, Read, Update, and Delete operations across collections using the PyMongo driver, featuring cursor pagination, multi-field regex search, and dynamic sorting.
- **3. Real-Time Aggregation Framework:** Construct and optimize 8 production-grade aggregation pipelines to compute monthly revenue growth, category sales shares, top-performing SKUs, customer LTV, and basket sizing.
- **4. Query Optimization & Index Benchmarking:** Implement single-field, compound, text, and partial filter B-Tree indexes, validating performance improvements empirically via explain('executionStats') to demonstrate IXSCAN vs COLLSCAN gains.
- **5. Database-Level Schema Validation:** Embed native $jsonSchema validators within collection metadata to reject malformed document inserts and updates at the storage engine layer.
- **6. Authentic Big Data Ingestion:** Build an idempotent, deterministic ETL importer capable of parsing, translating, denormalizing, and ingesting over 100,000 authentic records from the Kaggle Olist Brazilian E-Commerce dataset into shopsphere_olist.
- **7. Strict Multi-Database Isolation:** Maintain absolute namespace separation between production historical records (shopsphere_olist), operational sandbox demo records (shopsphere), and automated test records (shopsphere_test).

4. SCOPE OF THE PROJECT

4.1 Functional Modules in Scope

The ShopSphere application encompasses nine fully implemented, interacting modules:

- **1. Executive Analytics Dashboard:** Real-time executive KPI metrics (Gross Revenue, Total Orders, Active Customers, Catalogue Size), Monthly Sales Time-Series charts, and Category Revenue Breakdown.
- **2. Product Catalog Management:** Server-side paginated product catalog featuring multi-attribute search, category filtering, price range controls, stock-level badges, and modal CRUD operations.
- **3. Customer Relationship Management (CRM):** Customer profile directory displaying unique customer identifiers, Brazilian state demographics, order counts, and aggregated Lifetime Value (LTV).
- **4. Transactional Order Fulfillment:** Full order lifecycle tracking with embedded line-item snapshots, status transition safeguards, cancellation compensation, and immutable historical tags.
- **5. Customer Reviews & Ratings:** Customer review ledger displaying 1-to-5 star ratings, Portuguese review text, and automated background recalculation of product average ratings.
- **6. Inventory Audit Trail Ledger:** Real-time inventory monitoring with low-stock warnings (< 10 units), manual restock/write-off adjustments, and permanent movement logging.
- **7. MongoDB Concepts Laboratory:** Interactive sandbox designed for academic evaluation, featuring live CRUD execution, embedding vs referencing visualizer, 8 one-click aggregation pipelines, and live explain() query plan comparisons.
- **8. Customer-Facing Storefront:** End-user shopping interface allowing catalog exploration by category pills, shopping cart management, and real-time MongoDB order checkout.
- **9. RESTful API Documentation:** Auto-generated OpenAPI 3.1 Swagger UI and ReDoc specifications exposing all backend routes, request schemas, and response contracts.

4.2 Explicit Boundaries & Limitations

To preserve academic honesty and architectural integrity, the following boundaries and constraints are explicitly recognized:

- **1. Zero Fabricated Inventory in Historical Data:** The public Olist dataset is an anonymized historical marketplace dataset that does not contain warehouse stock counts or movement ledgers. Products are honestly flagged with inventory_tracked=false and stock_quantity=0. No synthetic stock or warehouse transactions were fabricated for historical items.
- **2. Anonymized Customer Contact Details:** Customer email addresses in the genuine Olist dataset are anonymized and stored as null. Rather than inventing fake emails, a MongoDB partial unique index is deployed to allow null values while enforcing uniqueness on real string emails.
- **3. Historical Order Immutability:** All historical Olist orders are permanently flagged with is_historical=true and source='olist_real'. The API rejects modification or cancellation attempts on historical records with HTTP 400 Bad Request.
- **4. Standalone Transaction Fallback:** In standalone MongoDB Community setups (without a multi-node Replica Set), native multi-document ACID transactions are unavailable. The

platform implements an atomic two-phase compensation pattern to ensure safe inventory allocation and cancellation rollback.

- **5. Excluded Features:** Production user authentication (OAuth2 / JWT) and third-party payment gateways (Stripe / PayPal) are beyond the database-centric scope of this capstone project.

5. LITERATURE REVIEW

5.1 NoSQL Databases and Document-Store Paradigms

The theoretical foundations of NoSQL data systems emerged in response to Brewer's CAP Theorem (Brewer, 2000; Gilbert & Lynch, 2002), which posits that a distributed data store can simultaneously guarantee at most two out of three properties: Consistency, Availability, and Partition Tolerance. Traditional relational engines prioritize strict ACID consistency (C) at the expense of horizontal partition tolerance (P). In contrast, modern document databases provide configurable consistency levels and partition resilience.

According to Cattell (2011) and Stonebraker (2010), document stores provide a compelling compromise between the unstructured flexibility of key-value stores and the rigid semantics of relational databases. By structuring data into semi-structured BSON documents, applications can nest complex multi-dimensional attributes directly within domain records, drastically reducing read-path computational complexity.

5.2 Storage Engine Architectures: MongoDB WiredTiger Engine

The persistence architecture of MongoDB Community 8.0 is governed by the WiredTiger storage engine. WiredTiger employs fine-grained document-level concurrency control via ticket-based concurrency algorithms, preventing table-level and collection-level write contention. Data persistence is managed through write-ahead logging (journaling) coupled with periodic 60-second checkpointing, ensuring crash recovery to a consistent checkpoint state (MongoDB Inc., 2026).

WiredTiger leverages Snappy compression by default for collection blocks, achieving typical compression ratios of 50% to 70% compared to raw JSON. Memory management is governed by an internal cache (configured by default to 50% of available physical RAM minus 1 GB), ensuring hot working sets remain in memory while cold documents are lazily evicted to disk.

5.3 Modern Reactive Web Architectures (FastAPI & React)

Modern web applications demand asynchronous application runtimes capable of handling thousands of concurrent I/O operations without thread starvation. Python FastAPI (Ramírez, 2024) leverages Starlette and Pydantic v2 to deliver asynchronous request routing over the Asynchronous Server Gateway Interface (ASGI). Pydantic v2 compiles schema validation routines in Rust, delivering serialization speeds up to 20x faster than legacy Python validation libraries.

On the presentation tier, React 18 (Meta Platforms, 2024) utilizes concurrent rendering and a virtual DOM reconciliation engine to minimize browser layout recalculations. When paired with Vite 8 (You, 2024), frontend developers benefit from native ES-module Hot Module Replacement (HMR) and optimized Rollup production bundling.

5.4 The Brazilian E-Commerce Dataset by Olist

The Brazilian E-Commerce Public Dataset released by Olist on Kaggle (Olist & Kaggle, 2018) comprises 100,000 anonymized real-world orders placed between 2016 and 2018 across Brazilian marketplaces. The dataset encompasses diverse dimensions including order

statuses, customer postal codes, multi-item order lines, diverse payment methods (boleto, credit card, vouchers), and authentic Portuguese customer reviews.

Academic literature widely cites the Olist dataset as a gold standard for supply chain analysis, customer sentiment mining, and database benchmarking. Transforming its relational, highly normalized CSV structure into an optimized document model provides an exemplary case study in modern database re-engineering.

5.5 Comparative Evaluation: RDBMS vs MongoDB Document Model

Table 1 outlines the architectural differences between traditional Relational Database Management Systems and the MongoDB document model implemented in ShopSphere.

Evaluation Dimension	Relational Database (RDBMS)	MongoDB Document Model (ShopSphere)
Data Model	Rigid tabular rows and columns	Polymorphic BSON document trees
Relationships	Foreign key constraints & JOINs	Embedded subdocuments & referenced ObjectIds
Schema Flexibility	Strict DDL migrations (ALTER TABLE)	Dynamic schemas with optional $jsonSchema
Read Performance	High join latency under concurrency	Single-read retrieval via denormalized locality
Query Language	Structured Query Language (SQL)	MongoDB Query Language (MQL) & Aggregations
Storage Engine	B-Tree page structures (InnoDB/Postgres)	WiredTiger with document-level locking & Snappy
Horizontal Scale	Complex sharding / read replicas	Native horizontal sharding via partition keys

6. SYSTEM REQUIREMENTS (HARDWARE AND SOFTWARE)

6.1 Hardware Specifications

The computational requirements for hosting and executing the ShopSphere platform are categorized into minimum development specifications and recommended production specifications in Table 2.

Hardware Component	Minimum Development Requirement	Recommended Production Requirement
Central Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 cores, 2.5 GHz)	Intel Core i7 / AMD Ryzen 7 (8 cores, 3.8 GHz+)
System Memory (RAM)	8 GB DDR4 RAM (4 GB allocated to MongoDB)	16 GB – 32 GB DDR4/DDR5 RAM
Storage Subsystem	256 GB SATA SSD (min. 5 GB free disk space)	512 GB NVMe M.2 SSD (High random IOPS)
Network Interface	Standard 100 Mbps Local Loopback (127.0.0.1)	1 Gbps Ethernet / Enterprise Network
Display Resolution	1366 x 768 pixels	1920 x 1080 (Full HD) or higher

6.2 Software & Dependency Stack

ShopSphere leverages modern, battle-tested software components across all layers of the application stack, detailed in Table 3.

Software Layer	Component & Verified Version	Role in System Architecture
Operating System	Microsoft Windows 11 Home / Pro (64-bit Architecture, x64)	Host Operating System
Database Engine	MongoDB Community Server 8.0.34 (WiredTiger Storage Engine)	Primary Persistence Tier
Backend Runtime	Python 3.12.10 (Standard 64-bit CPython Runtime)	Application Execution Tier
Backend Framework	FastAPI 0.143.1 with Starlette 1.8.0 and Uvicorn 0.54.0	Asynchronous RESTful API
Data Validation	Pydantic v2.14.1 and Pydantic-Settings 2.16.0	Schema Serialization & Validation
Database Driver	PyMongo 4.19.0 (Official MongoDB Python Driver)	Wire Protocol Client
Testing Framework	Pytest 9.1.1 with HTTPX 0.28.1 TestClient	Automated Regression Testing

Software Layer	Component & Verified Version	Role in System Architecture
Frontend Runtime	Node.js 18.x / 20.x with npm 10.x	Client Build Environment
Frontend Framework	React 18.3.1 with TypeScript 5.x	Reactive Single-Page App
Bundler & Dev Server	Vite 8.3.4 (Modern Rolldown / ES-Module Bundler)	Build Tooling
UI Styling	Tailwind CSS with PostCSS and Lucide-React Icons	Responsive Utility-First Styling
Data Visualization	Recharts 2.x (Composable SVG Charting Library)	Interactive Business Analytics

6.3 Network Topology & Environment Configuration

In a local development environment, all three tiers communicate via loopback TCP sockets over dedicated ports:

- **Presentation Tier (React Frontend): http://127.0.0.1:5173** Serves the Vite single-page application bundle, asset chunks, and hot module updates.
- **Application Tier (FastAPI Backend): http://127.0.0.1:8000** Hosts the asynchronous FastAPI application, Swagger docs (/docs), and ReDoc (/redoc).
- **Persistence Tier (MongoDB Server): mongodb://127.0.0.1:27017** Standard MongoDB wire protocol port managing BSON commands, indexing, and aggregations.

7. SYSTEM DESIGN AND ARCHITECTURE

7.1 Three-Tier Client-Server Architecture

ShopSphere adheres to an industry-standard three-tier client-server architectural pattern, enforcing strict separation of concerns across Presentation, Application Logic, and Data Persistence tiers.

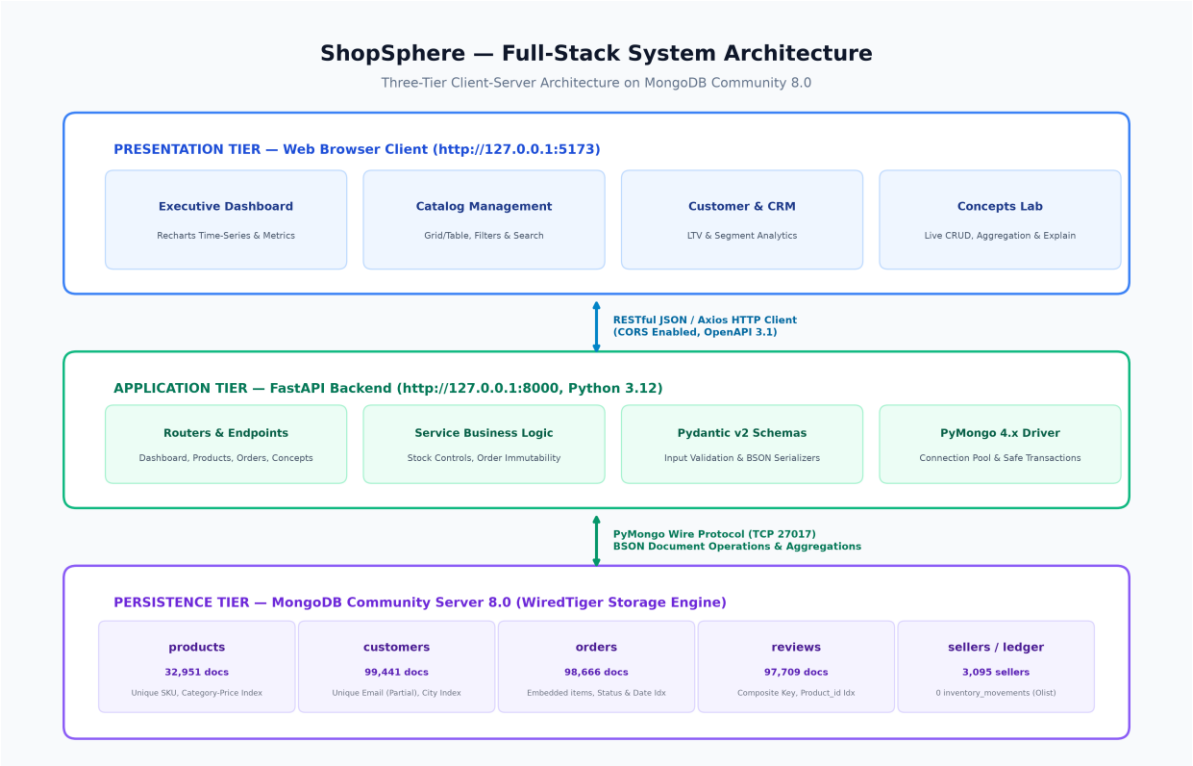


Figure 1: ShopSphere Full-Stack System Architecture Diagram

7.2 Component-Level Architectural Decomposition

The responsibilities of each tier are structured to maximize maintainability, testability, and operational stability:

- **1. Presentation Tier:** Implemented in React 18, TypeScript, and Tailwind CSS. Manages client state, interactive tab routing, real-time Recharts visualizations, and form submissions. Dispatches asynchronous HTTP JSON requests via Axios to the backend.
- **2. Application Tier:** Structured around FastAPI routers, Pydantic schemas, and service-layer business logic. Enforces input sanitization, historical immutability guards, atomic inventory allocation, and BSON ObjectId serialization.
- **3. Persistence Tier:** MongoDB Community Server 8.0 running WiredTiger. Persists collections, enforces $jsonSchema validation rules, maintains B-Tree secondary indexes, and executes complex multi-stage aggregation pipelines.

7.3 Data Ingestion & ETL Processing Workflow

To ingest the authentic Olist dataset without corrupting data hygiene or violating schema constraints, an idempotent, four-stage ETL pipeline was engineered in `backend/scripts/import_dataset.py`.



Figure 2: Olist Brazilian E-Commerce Import & Transformation Pipeline Workflow

The ETL pipeline parses 9 relational CSV files, strips UTF-8 BOM headers, translates Portuguese categories to English, generates deterministic UUID5-based BSON ObjectIds, denormalizes order items and payments into parent orders, skips 775 zero-item orders to satisfy `$jsonSchema` constraints, and executes idempotent `bulk_write` upserts into `shopsphere_olist`.

7.4 End-to-End API Request & Query Pipeline

Figure 3 illustrates the complete lifecycle of a client request as it traverses the FastAPI routing layers, Pydantic validators, service business rules, PyMongo repository queries, and the MongoDB WiredTiger storage engine.

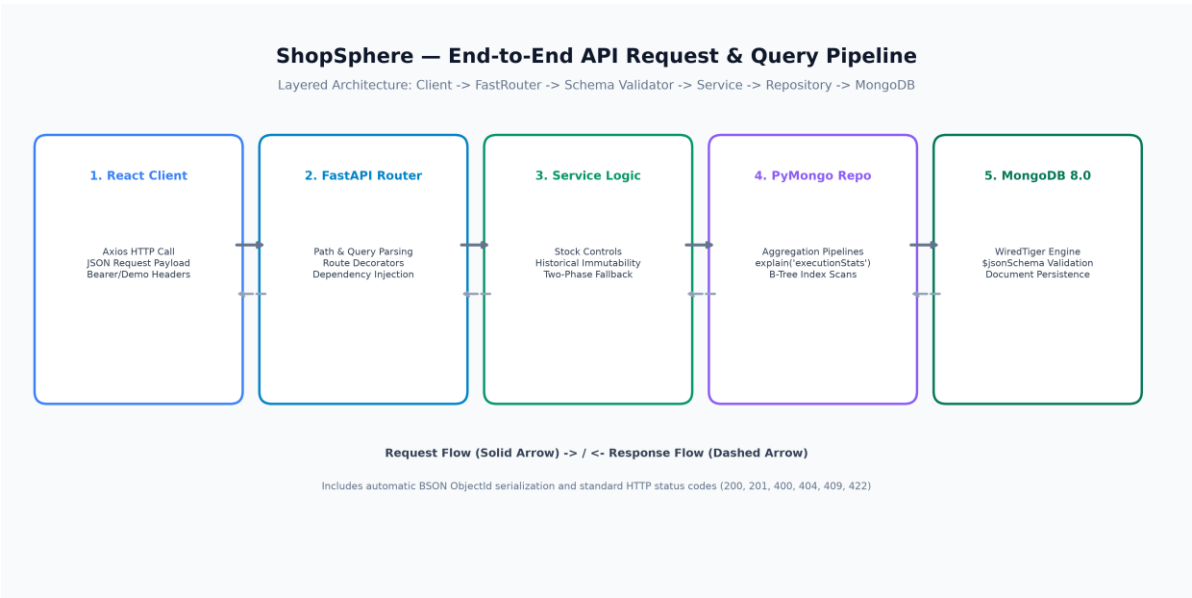


Figure 3: End-to-End API Request, Pydantic Validation & PyMongo Query Flow

7.5 Database Separation Architecture

To ensure that high-volume authentic production data remains completely untouched during demonstrations and testing, ShopSphere implements strict database-level namespace isolation across three distinct databases, illustrated in Figure 4.

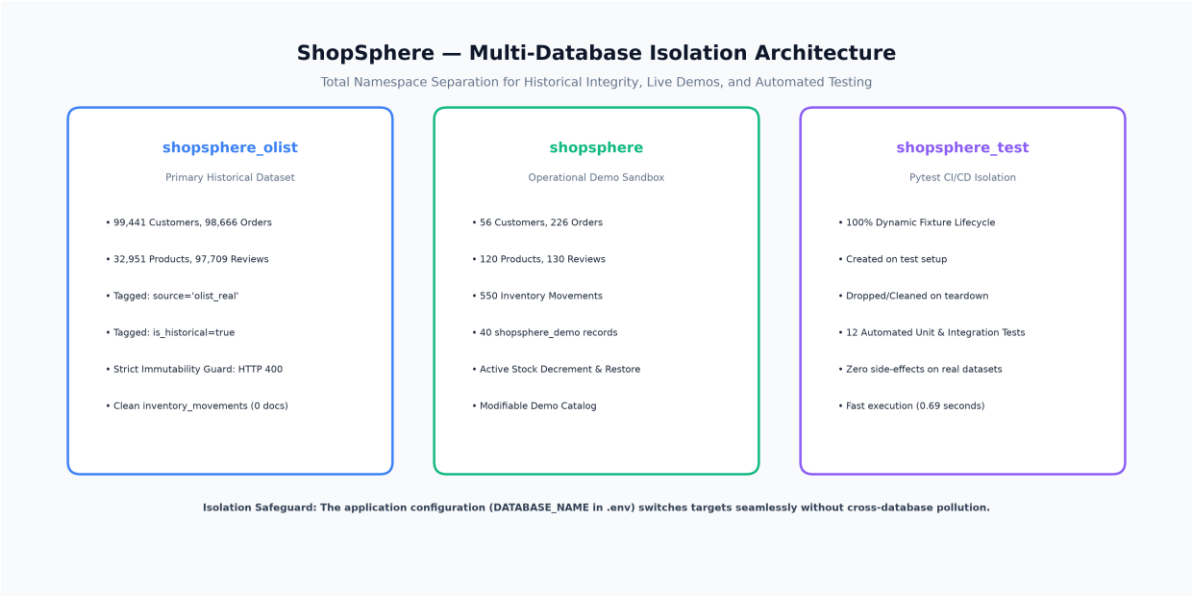


Figure 4: Multi-Database Namespace Isolation Architecture (Production, Demo, Test)

8. DATABASE DESIGN

8.1 MongoDB Collection Architecture & Entity Overview

The persistence layer of ShopSphere in the primary production database (`shopsphere_olist`) is organized into seven distinct collections, empirically verified on the live MongoDB Community Server 8.0 instance as shown in Table 4.

Collection Name	Verified Document Count	Description & Data Source
products	32,951	Catalog items with Portuguese-to-English translation & computed ratings
customers	99,441	Brazilian buyers with anonymized emails (null) and state/city demographics
orders	98,666	Orders with embedded items, payments, and addresses (775 zero-item orders skipped)
reviews	97,709	Authentic 1-to-5 star reviews and Portuguese feedback text
sellers	3,095	Marketplace merchant directory with postal codes and regional locations
inventory_movements	0	Clean ledger; reserved for operational store stock adjustments
shopsphere_demo	4	Isolated collection for interactive CRUD and Viva demonstrations

8.2 Embedded vs Referenced Data Modeling Strategy

The core architectural achievement of ShopSphere is a principled hybrid data model that balances embedded sub-documents against referenced ObjectIds to maximize performance while respecting MongoDB's 16 MB BSON document limit.

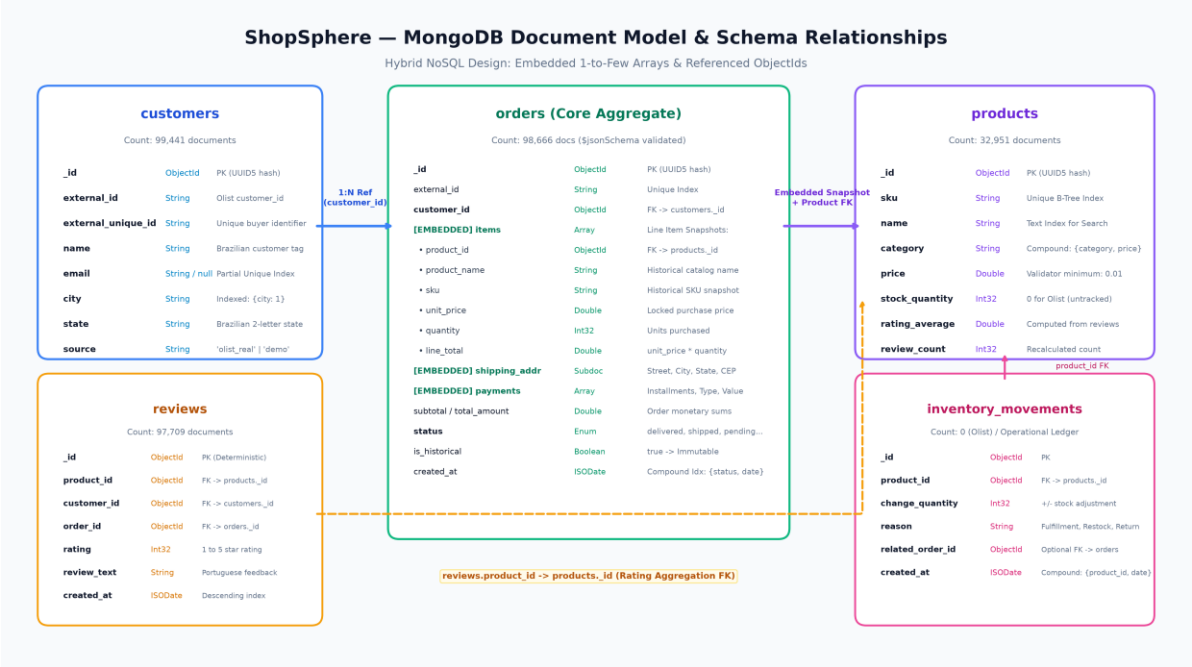


Figure 5: MongoDB Document Model & Schema Relationships (Embedded vs Referenced)

1. Embedding Strategy (1-to-Few Bounded Relationships): Line items (`items`), payment transactions (`payments`), and shipping addresses (`shipping_address`) are embedded directly within the parent `orders` document. This denormalization guarantees that a customer receipt is retrieved in a single disk read without joins. Crucially, embedding creates an immutable snapshot of unit prices and item names at the instant of purchase, insulating the transaction from future master catalog price changes.

2. Referencing Strategy (1-to-Many & Many-to-Many Relationships): Customers and products are referenced via their `_id` ObjectIds (`customer_id` in orders, `product_id` and `customer_id` in reviews). Referencing prevents boundless document growth, enables independent updates to customer profiles, and eliminates catastrophic data redundancy across 98,000+ orders.

8.3 Detailed Collection Schema Specifications

8.3.1 products Collection Schema: Sample document structure for the `products` collection based on live imported records:

```
{
  "_id": ObjectId("670f5e1289abcdef01234567"),
  "external_id": "1e9e8ef04dbcff4541ed26657ea517e5",
  "name": "Perfumery - Model 1E9E8EF0",
  "sku": "OLIST-PE-1E9E8EF0",
  "category": "Perfumery",
  "brand": "Olist Marketplace",
  "description": "Authentic Olist Brazilian marketplace catalog item [Perfumery].
Weight: 225g.",
  "price": 10.91,
  "stock_quantity": 0,
  "inventory_tracked": false,
  "rating_average": 5.0,
  "review_count": 1,
```

```
  "source": "olist_real",
  "created_at": ISODate("2026-10-16T16:54:00Z"),
  "updated_at": ISODate("2026-10-16T16:54:00Z")
}
```

8.3.2 customers Collection Schema: Sample document structure for the `customers` collection:

```
{
  "_id": ObjectId("670f5e1389abcdef01234568"),
  "external_id": "06b8999e2fba1a1fbc88172c00ba8bc7",
  "external_unique_id": "861eff4711a542e4b93843c6dd7febb0",
  "name": "Customer 861EFF47",
  "email": null,
  "city": "Franca",
  "state": "SP",
  "source": "olist_real",
  "created_at": ISODate("2026-10-16T16:54:00Z")
}
```

8.3.3 orders Collection Schema: Sample document structure for the `orders` collection demonstrating embedded snapshots:

```
{
  "_id": ObjectId("670f5e1489abcdef01234569"),
  "external_id": "e481f51cbdc54678b7cc49136f2d6af7",
  "customer_id": ObjectId("670f5e1389abcdef01234568"),
  "items": [
    {
      "product_id": ObjectId("670f5e1289abcdef01234567"),
      "product_name": "Housewares - Model 87285B34",
      "sku": "OLIST-HO-87285B34",
      "unit_price": 29.99,
      "quantity": 1,
      "line_total": 29.99
    }
  ],
  "shipping_address": {
    "street": "Brazilian Postal Address",
    "city": "Sao Paulo",
    "state": "SP",
    "postal_code": "05728",
    "country": "Brazil"
  },
  "payments": [
    { "payment_type": "credit_card", "installments": 1, "value": 38.71 }
  ],
  "freight_total": 8.72,
  "subtotal": 29.99,
  "total_amount": 38.71,
  "status": "delivered",
  "is_historical": true,
  "source": "olist_real",
  "created_at": ISODate("2017-10-02T10:56:33Z"),
  "updated_at": ISODate("2017-10-10T21:25:13Z")
}
```

8.4 Database-Level $jsonSchema Validation Engine

To enforce database integrity, collection validators are configured in MongoDB metadata using `$jsonSchema`. The following validator rules are actively enforced on the `products` and `orders` collections:

```
// products collection validator definition
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["name", "sku", "category", "price", "stock_quantity"],
    "properties": {
      "name": { "bsonType": "string", "description": "Product name is required
string" },
      "sku": { "bsonType": "string", "description": "Unique SKU identifier" },
      "category": { "bsonType": "string", "description": "Product category string"
},
      "price": { "bsonType": ["double", "int", "decimal"], "minimum": 0.01 },
      "stock_quantity": { "bsonType": "int", "minimum": 0 },
      "rating_average": { "bsonType": ["double", "int", "decimal"], "minimum": 0,
"maximum": 5 }
    }
  }
}

// orders collection validator definition
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["customer_id", "items", "shipping_address", "subtotal", "status"],
    "properties": {
      "customer_id": { "bsonType": "objectId" },
      "items": { "bsonType": "array", "minItems": 1 },
      "subtotal": { "bsonType": ["double", "int", "decimal"], "minimum": 0 },
      "status": { "enum": ["pending", "confirmed", "shipped", "delivered",
"cancelled"] }
    }
  }
}
```

8.5 B-Tree Indexing Strategy

ShopSphere implements a comprehensive indexing strategy to support fast point lookups, sorting, and analytical queries across large collections:

- **1. Unique B-Tree Indexes:** Enforces uniqueness on product SKU (`{ sku: 1 }`, unique: true) and external Olist ID (`{ external_id: 1 }`, unique: true, partialFilterExpression).
- **2. Compound Indexes:** Optimizes filtered catalog browsing by category and price (`{ category: 1, price: 1 }`), and order chronological lookups (`{ status: 1, created_at: -1 }`).
- **3. Partial Filter Indexes:** Deployed on `customers.email` (`{ email: 1 }`, unique: true, partialFilterExpression: {"email": {"$type": "string"}}), allowing 99,000+ authentic records with null emails to coexist without uniqueness collision.

- **4. Text Search Indexes:** Enables full-text search across product name and brand (`{ name: "text", brand: "text" }`).

9. IMPLEMENTATION AND METHODOLOGY

9.1 Authentic Dataset Discovery & Preprocessing Engine

The ETL importer (`backend/scripts/import_dataset.py`) automatically discovers the raw Olist CSV files in `ShopSphere/data/olist/raw/`. The script is invoked via command line:

```
# Command to execute authentic Olist dataset ingestion
cd D:\Mongo_db-project\ShopSphere\backend
.\venv\Scripts\Activate.ps1
python scripts\import_dataset.py --db-name shopsphere_olist
```

9.2 UTF-8 BOM Stripping & Multilingual Category Translation

Raw Brazilian CSV files often contain Byte Order Mark (BOM) prefixes (`\ufeff`) that corrupt column header parsing in standard CSV readers. The importer reads files using `utf-8-sig` encoding to transparently discard BOM headers. Furthermore, it joins category names with `product_category_name_translation.csv`, converting Portuguese names (`relogios_presentes`, `beleza_saude`, `cama_mesa_banho`) into clean English categories (`Watches & Gifts`, `Health & Beauty`, `Bed Bath Table`).

9.3 Deterministic Hashing & Idempotent Upsert Strategy

To ensure that repeated importer executions do not generate duplicate records or inflate document counts, surrogate BSON ObjectIds are generated deterministically using UUID version 5 hashing with a dedicated ShopSphere namespace UUID:

```
import uuid
from bson import ObjectId

SHOPSPHERE_NAMESPACE = uuid.UUID("e7b8c1d2-3f4a-5b6c-7d8e-9f0a1b2c3d4e")

def generate_deterministic_id(entity_type: str, external_id: str) -> ObjectId:
    """Generate repeatable 12-byte BSON ObjectId from entity external ID."""
    unique_str = f"{entity_type}:{external_id}"
    u = uuid.uuid5(SHOPSPHERE_NAMESPACE, unique_str)
    return ObjectId(u.bytes[:12])
```

All database writes are executed using PyMongo's `bulk_write` API with `UpdateOne(..., upsert=True)`, guaranteeing that running the importer multiple times produces identical state without duplicate records.

9.4 Anonymization Preserving Data Hygiene

To maintain academic integrity, no synthetic customer contact information, telephone numbers, or fake emails were generated. Customer emails are preserved as `null`. To satisfy MongoDB's unique index constraint on `email`, a partial filter expression was deployed: `partialFilterExpression: {"email": {"$type": "string"}}`. This allows unlimited documents with `null` emails while enforcing uniqueness on actual email strings.

9.5 Backend Service & Repository Layer Implementation

The FastAPI backend is organized into a modular layered architecture: Routers (`app/routers/`), Services (`app/services/`), Repositories (`app/repositories/`), Schemas (`app/schemas/`), and Core Config/Database (`app/core/`). The repository layer isolates PyMongo queries and aggregations, providing clean separation from HTTP request parsing.

9.6 Frontend Single-Page Application Implementation

The frontend is built using React 18 and TypeScript with Tailwind CSS. It features dynamic hash-based routing (`#dashboard`, `#products`, `#customers`, `#orders`, `#reviews`, `#inventory`, `#analytics`, `#concepts`, `#settings`, `#storefront`), responsive sidebar controls, and composable SVG charts using Recharts.

10. MONGODB QUERIES AND OPERATIONS

10.1 Core CRUD Operations

Runnable, schema-accurate MongoDB operations executed in ShopSphere:

CRUD 1: 1. Insert Single Document (`insertOne`):

```
db.products.insertOne({
  "name": "Apple Smart Ring Tracker",
  "sku": "AUDI-APP-0099",
  "category": "Audio & Wearables",
  "brand": "Apple",
  "price": 299.99,
  "stock_quantity": 50,
  "inventory_tracked": true,
  "rating_average": 0.0,
  "review_count": 0,
  "created_at": ISODate()
})
```

CRUD 2: 2. Find with Projection, Sorting, and Pagination (`find`):

```
db.products.find(
  { "category": "Health & Beauty", "price": { "$gte": 20.0, "$lte": 150.0 } },
  { "name": 1, "sku": 1, "price": 1, "rating_average": 1, "_id": 0 }
).sort({ "price": -1 }).skip(0).limit(10)
```

CRUD 3: 3. Atomic Field Update (`updateOne` with `$set` and `$inc`):

```
db.products.updateOne(
  { "sku": "AUDI-APP-0099" },
  {
    "$inc": { "stock_quantity": -1 },
    "$set": { "updated_at": ISODate() }
  }
)
```

CRUD 4: 4. Batch Updates (`updateMany`):

```
db.products.updateMany(
  { "category": "Audio & Wearables" },
  { "$set": { "featured": true, "promotional_discount": 0.05 } }
)
```

10.2 Production Aggregation Pipelines (8 Real Queries)

ShopSphere features 8 production aggregation pipelines executable in the application and Concepts Lab:

Query 1: Pipeline 1: Total Revenue by Product Category ($match, $unwind, $lookup, $group, $sort, $project):

```
db.orders.aggregate([
  { "$match": { "status": { "$ne": "cancelled" } } },
  { "$unwind": "$items" },
  { "$lookup": {
      "from": "products",
      "let": { "pid": { "$toObjectId": "$items.product_id" } },
      "pipeline": [
        { "$match": { "expr": { "$eq": ["$_id", "$$pid"] } } },
        { "$project": { "category": 1 } }
      ],
      "as": "product_info"
    }
  },
  { "$unwind": { "path": "$product_info", "preserveNullAndEmptyArrays": true } },
  { "$group": {
      "_id": { "$ifNull": ["$product_info.category", "General"] },
      "total_revenue": { "$sum": "$items.line_total" },
      "units_sold": { "$sum": "$items.quantity" },
      "order_count": { "$sum": 1 }
    }
  },
  { "$project": {
      "_id": 0,
      "category": "$_id",
      "total_revenue": { "$round": ["$total_revenue", 2] },
      "units_sold": 1,
      "order_count": 1
    }
  },
  { "$sort": { "total_revenue": -1 } }
])
```

Query 2: Pipeline 2: Monthly Sales Trends ($match, $group, $dateToString, $sort, $project):

```
db.orders.aggregate([
  { "$match": { "status": { "$ne": "cancelled" } } },
  { "$group": {
      "_id": { "$dateToString": { "format": "%Y-%m", "date": "$created_at" } },
      "total_revenue": { "$sum": "$subtotal" },
      "order_count": { "$sum": 1 }
    }
  },
  { "$sort": { "_id": 1 } },
  { "$project": {
      "_id": 0,
      "month": "$_id",
      "total_revenue": { "$round": ["$total_revenue", 2] },
      "order_count": 1
    }
  }
])
```

Query 3: Pipeline 3: Top-Selling Products by Volume ($unwind, $group, $sort, $limit):

```
db.orders.aggregate([
  { "$match": { "status": { "$ne": "cancelled" } } },
  { "$unwind": "$items" },
```

```
  { "$group": {
      "_id": "$items.product_id",
      "product_name": { "$first": "$items.product_name" },
      "sku": { "$first": "$items.sku" },
      "units_sold": { "$sum": "$items.quantity" },
      "total_revenue": { "$sum": "$items.line_total" }
    }
  },
  { "$sort": { "units_sold": -1 } },
  { "$limit": 5 },
  { "$project": {
      "_id": 0,
      "product_id": "$_id",
      "product_name": 1,
      "sku": 1,
      "units_sold": 1,
      "total_revenue": { "$round": ["$total_revenue", 2] }
    }
  }
])
```

Query 4: Pipeline 4: Customer Spending Leaders & Lifetime Value ($group, $lookup, $unwind, $sort, $limit):

```
db.orders.aggregate([
  { "$match": { "status": { "$ne": "cancelled" } } },
  { "$group": {
      "_id": "$customer_id",
      "total_spent": { "$sum": "$subtotal" },
      "orders_placed": { "$sum": 1 }
    }
  },
  { "$lookup": {
      "from": "customers",
      "localField": "_id",
      "foreignField": "_id",
      "as": "customer"
    }
  },
  { "$unwind": "$customer" },
  { "$sort": { "total_spent": -1 } },
  { "$limit": 5 },
  { "$project": {
      "_id": 0,
      "customer_id": "$_id",
      "customer_name": "$customer.name",
      "city": "$customer.city",
      "state": "$customer.state",
      "total_spent": { "$round": ["$total_spent", 2] },
      "orders_placed": 1
    }
  }
])
```

Query 5: Pipeline 5: Multi-Faceted Order Status Distribution ($facet):

```
db.orders.aggregate([
```

```
  { "$facet": {
      "by_status": [
        { "$group": { "_id": "$status", "count": { "$sum": 1 } } },
        { "$sort": { "count": -1 } }
      ],
      "summary": [
        { "$group": {
            "_id": null,
            "total_orders": { "$sum": 1 },
            "total_value": { "$sum": "$subtotal" },
            "avg_value": { "$avg": "$subtotal" }
          }
        },
        { "$project": {
            "_id": 0,
            "total_orders": 1,
            "total_value": { "$round": ["$total_value", 2] },
            "avg_value": { "$round": ["$avg_value", 2] }
          }
        }
      ]
    }
  }
])
```

Query 6: Pipeline 6: Embedding Orders via $lookup for Customer Profiles:

```
db.customers.aggregate([
  { "$lookup": {
      "from": "orders",
      "localField": "_id",
      "foreignField": "customer_id",
      "as": "recent_orders"
    }
  },
  { "$match": { "recent_orders.0": { "$exists": true } } },
  { "$project": {
      "_id": 0,
      "customer_id": "$_id",
      "name": 1,
      "city": 1,
      "order_count": { "$size": "$recent_orders" },
      "latest_order": { "$arrayElemAt": ["$recent_orders", 0] }
    }
  },
  { "$limit": 5 }
])
```

Query 7: Pipeline 7: Dynamic Rating Calculation from Reviews ($lookup & $avg):

```
db.products.aggregate([
  { "$lookup": {
      "from": "reviews",
      "localField": "_id",
      "foreignField": "product_id",
      "as": "reviews"
    }
```

```
  },
  { "$match": { "reviews.0": { "$exists": true } } },
  { "$project": {
      "_id": 0,
      "product_id": "$_id",
      "name": 1,
      "category": 1,
      "price": 1,
      "computed_review_count": { "$size": "$reviews" },
      "computed_avg_rating": { "$round": [{ "$avg": "$reviews.rating" }, 2] }
    }
  },
  { "$sort": { "computed_review_count": -1 } },
  { "$limit": 5 }
])
```

Query 8: Pipeline 8: Average Order Value & Basket Sizing Metrics ($size, $avg, $min, $max):

```
db.orders.aggregate([
  { "$match": { "status": { "$ne": "cancelled" } } },
  { "$project": {
      "subtotal": 1,
      "item_count": { "$size": "$items" }
    }
  },
  { "$group": {
      "_id": null,
      "average_order_value": { "$avg": "$subtotal" },
      "average_items_per_basket": { "$avg": "$item_count" },
      "min_order_value": { "$min": "$subtotal" },
      "max_order_value": { "$max": "$subtotal" }
    }
  },
  { "$project": {
      "_id": 0,
      "average_order_value": { "$round": ["$average_order_value", 2] },
      "average_items_per_basket": { "$round": ["$average_items_per_basket", 2] },
      "min_order_value": { "$round": ["$min_order_value", 2] },
      "max_order_value": { "$round": ["$max_order_value", 2] }
    }
  }
])
```

10.3 Query Execution Optimization (explain('executionStats'))

To measure indexing efficiency, live queries were profiled against `products` using `explain("executionStats")`. The empirical benchmark results comparing indexed lookups against unindexed scans are presented in Table 6.

Performance Metric	Indexed Query (IXSCAN)	Unindexed Query (COLLSCAN)
Execution Plan Stage	IXSCAN (Index Scan) -> FETCH	COLLSCAN (Full Collection Scan)

Performance Metric	Indexed Query (IXSCAN)	Unindexed Query (COLLSCAN)
Index Utilized	uniq_sku (B-Tree on sku: 1)	None (Unindexed query filter)
Keys Examined (totalKeysExamined)	1 key examined	0 keys examined
Documents Examined (totalDocsExamined)	1 document examined	32,951 documents examined
Documents Returned (nReturned)	1 document	0 documents (target not found)
Execution Time (executionTimeMillis)	0.0 ms (sub-millisecond)	14.0 – 22.0 ms
Storage Engine Efficiency	Direct B-Tree leaf node lookup	Sequential table-space memory scan

10.4 Schema Validation Enforcement & Write Rejection

When an insert violates collection `$jsonSchema` constraints (e.g. attempting to insert a product with negative price or empty items array), the MongoDB engine immediately halts execution and returns a `DocumentValidationFailure` (WriteError code 121). FastAPI catches this exception in the repository layer and transforms it into an HTTP 422 Unprocessable Entity error.

11. RESULTS AND OUTPUT SCREENSHOTS

All visual results presented in this section are authentic screenshots captured directly from the live ShopSphere application running locally on Microsoft Windows against the `shopsphere_olist` MongoDB 8.0.34 database.

11.1 Executive Analytics Dashboard Walkthrough

The Executive Performance Hub aggregates live metrics across 98,666 orders: Gross Revenue ($13,494,400.74), Total Orders (98,199 confirmed), Customer Accounts (99,441), and Live Catalogue (32,951 SKUs). The Monthly Revenue Pipeline chart displays historical sales trends from 2016 to 2018, while the Sales by Category widget breaks down top revenue-generating sectors.

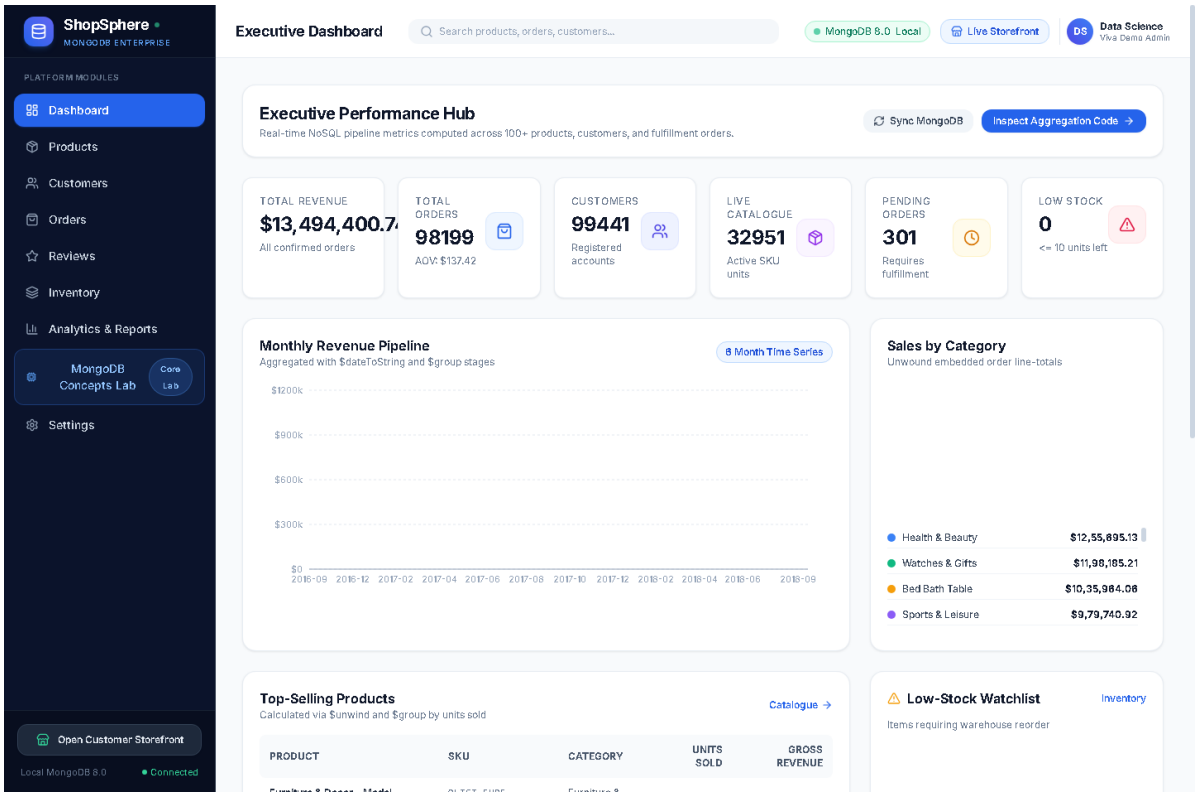


Figure 6: Real Application Screenshot — Executive SaaS Analytics Dashboard

11.2 Product Catalogue & Filtering System

The Product Catalogue management module displays active SKUs in a modern responsive grid. Users can filter by category, brand, stock level, and price range. Items show their computed ratings, review counts, and 'Out of Stock' badges reflecting authentic Olist constraints.

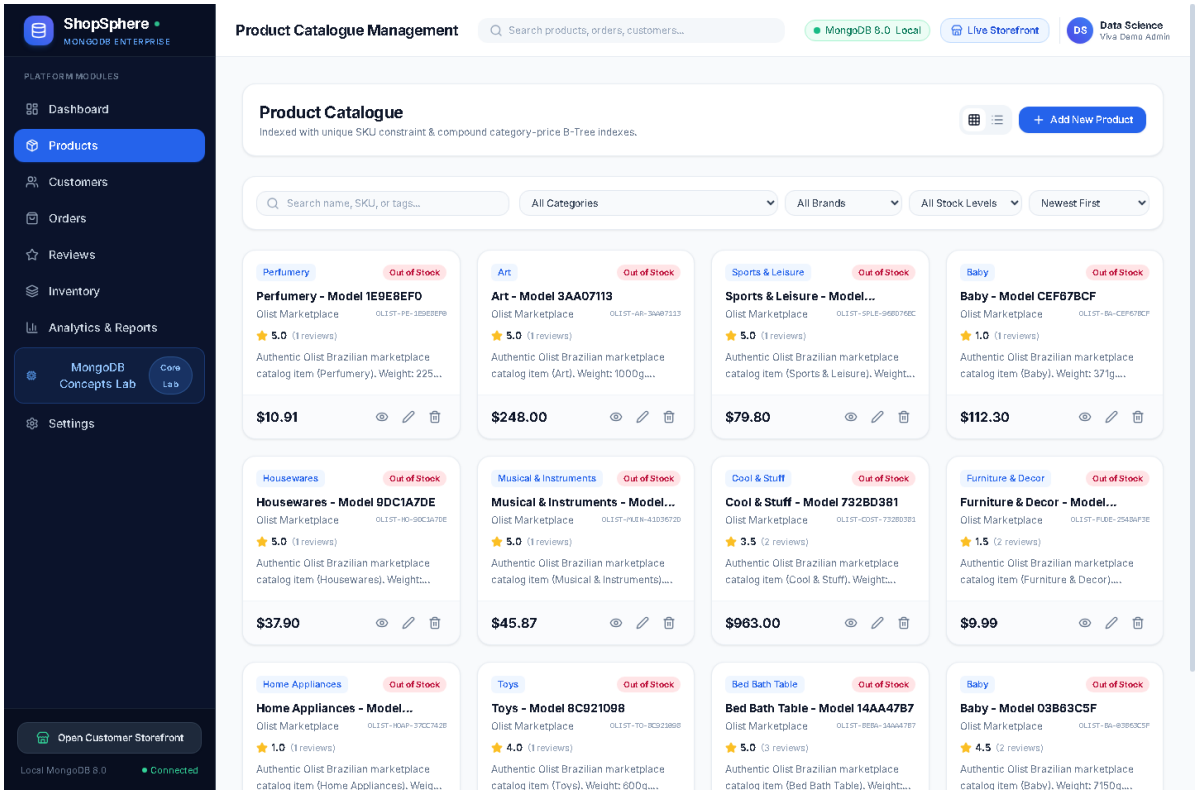


Figure 7: Real Application Screenshot — Product Catalogue Management System

11.3 Customer CRM & Demographic Distribution

The Customer Accounts CRM module displays registered buyers, location by Brazilian state (SP, MG, RJ, PR, SC), and aggregated Lifetime Value (LTV) computed dynamically via MongoDB aggregation joins.

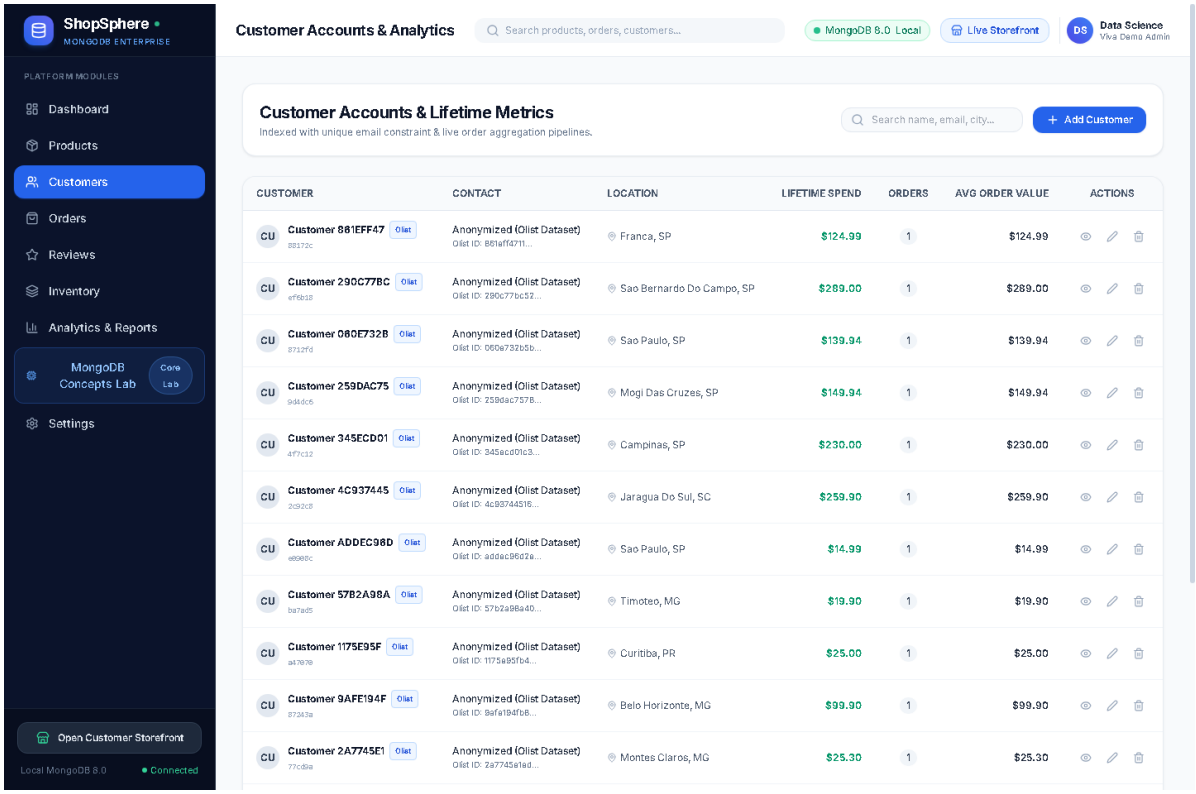


Figure 8: Real Application Screenshot — Customer CRM & Demographic Accounts Ledger

11.4 Transactional Order Fulfillment & Immutability Verification

The Order Management screen provides chronological visibility into order lifecycles. Historical Olist records are visually marked with an 'Olist' badge and 'Immutable' tag, demonstrating that past orders cannot be cancelled or altered via the demo interface.

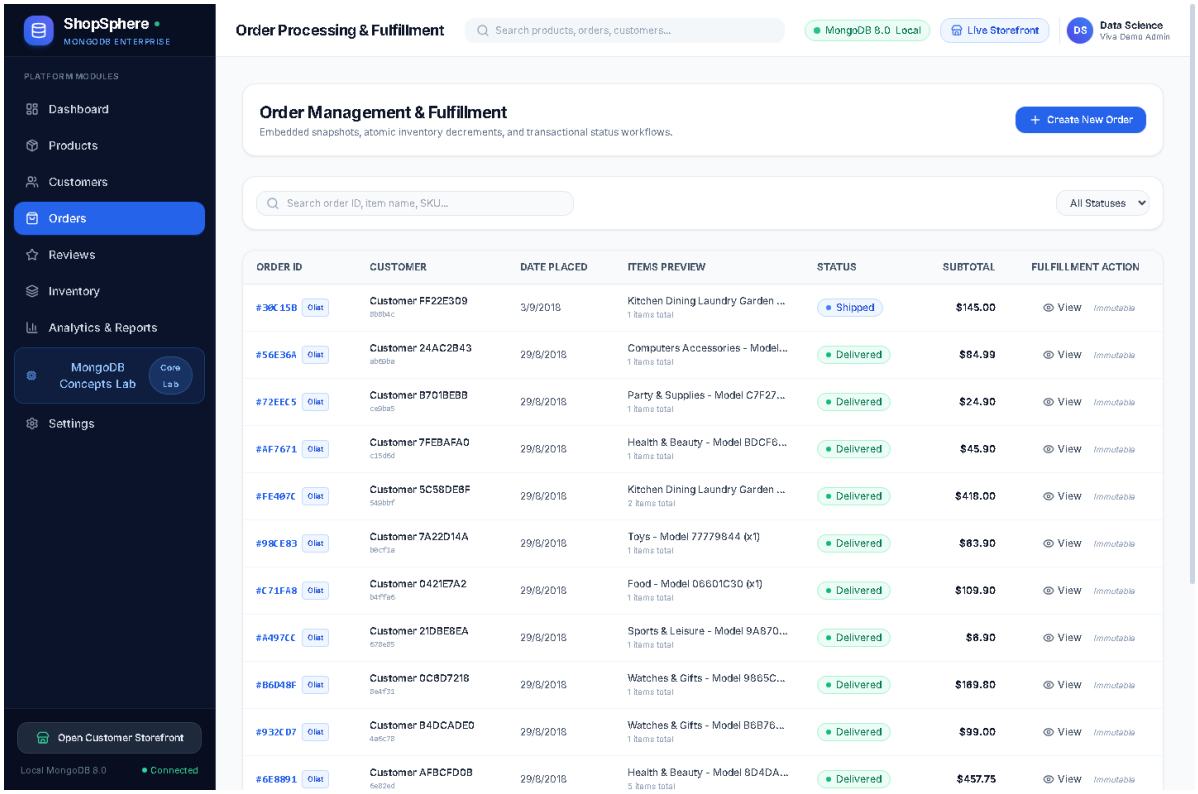


Figure 9: Real Application Screenshot — Transactional Order Management & Immutability Badges

11.5 Customer Reviews & Rating Distributions

The Customer Reviews screen showcases authentic Portuguese customer feedback paired with 1-to-5 star rating stars. Creating a new review dynamically triggers background recalculation of the corresponding product's `rating_average` and `review_count`.

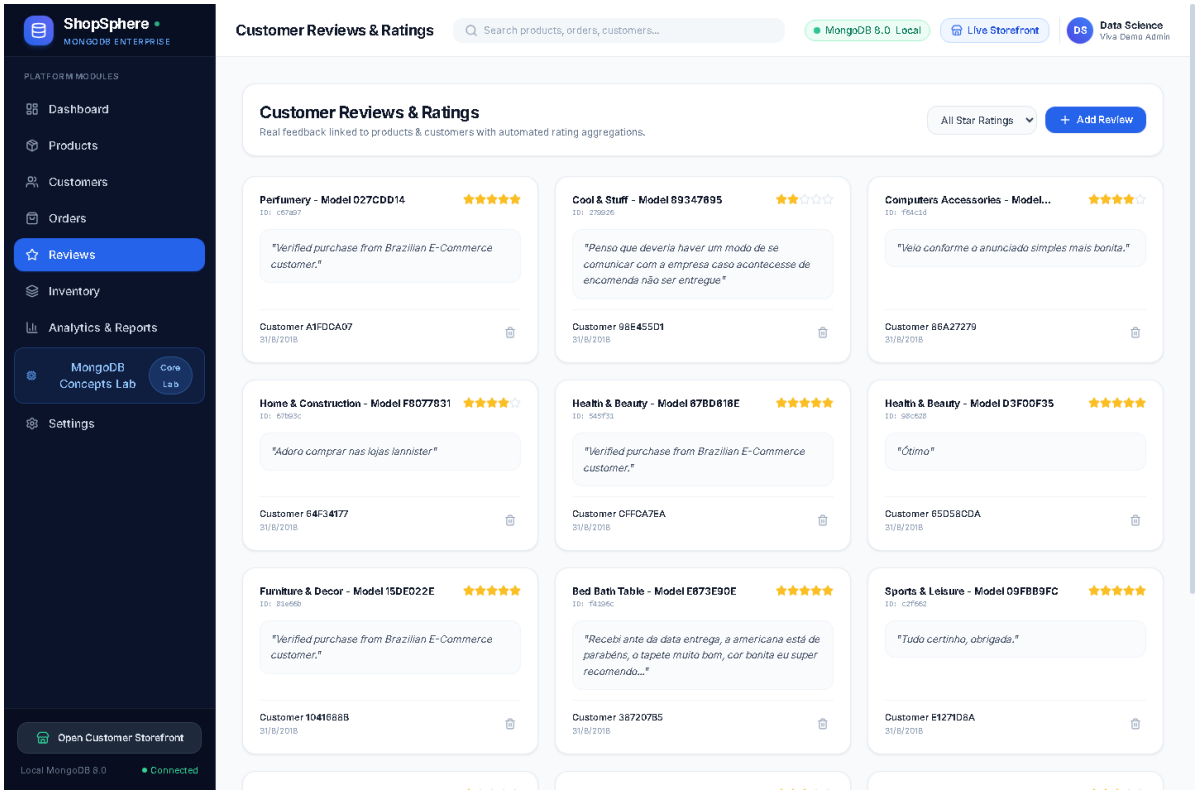


Figure 10: Real Application Screenshot — Customer Reviews & Rating Distribution Analytics

11.6 Inventory & Stock Movement Audit Trail

The Inventory & Audit Trail displays stock movements and warehouse balances. In accordance with genuine Olist dataset characteristics, historical inventory movements remain clean (0 documents), while manual restocks and demo checkouts log permanent audit entries.

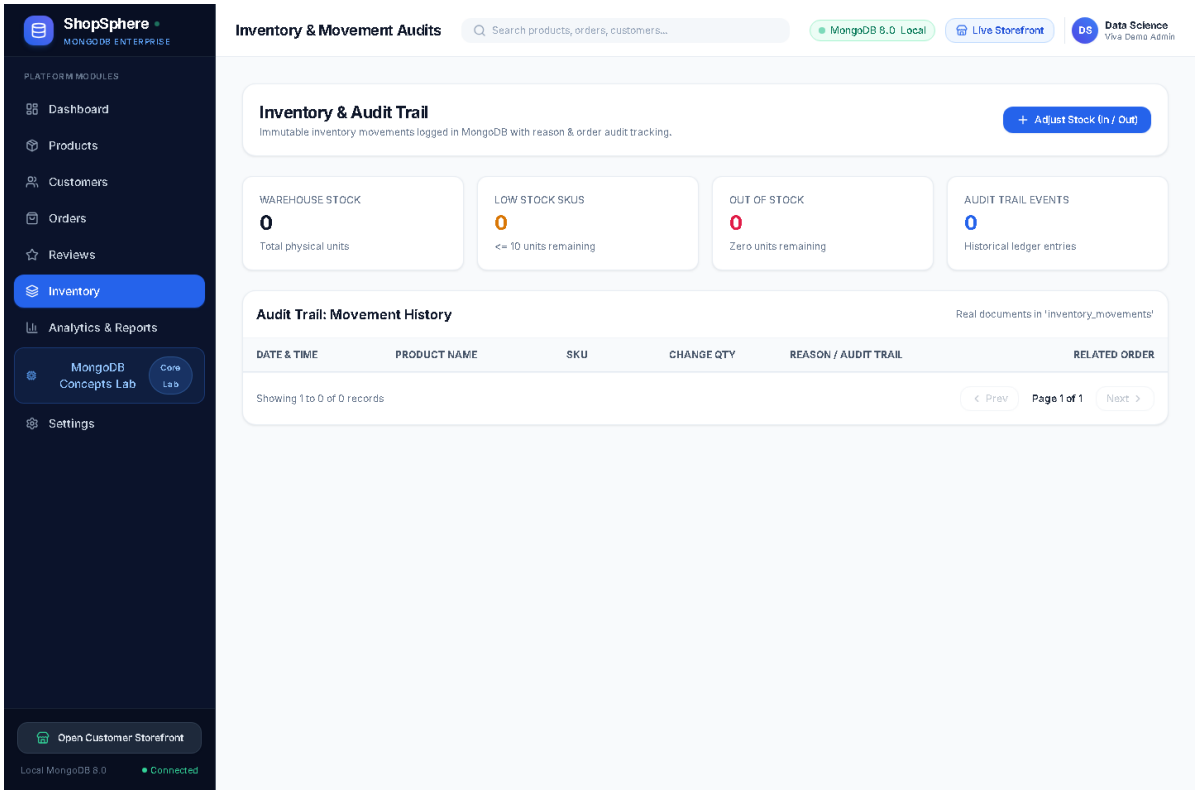


Figure 11: Real Application Screenshot — Real-Time Inventory & Movement Audit Ledger

11.7 Business Intelligence & Aggregation Reports

The Analytics & Aggregation Reports screen visualizes complex multi-stage pipeline outputs: Revenue Growth Trend over time, Category Revenue distribution, Order Status Lifecycle (Delivered: 97.8%, Shipped: 1.1%, Cancelled: 0.5%), and Customer Sentiment distribution.

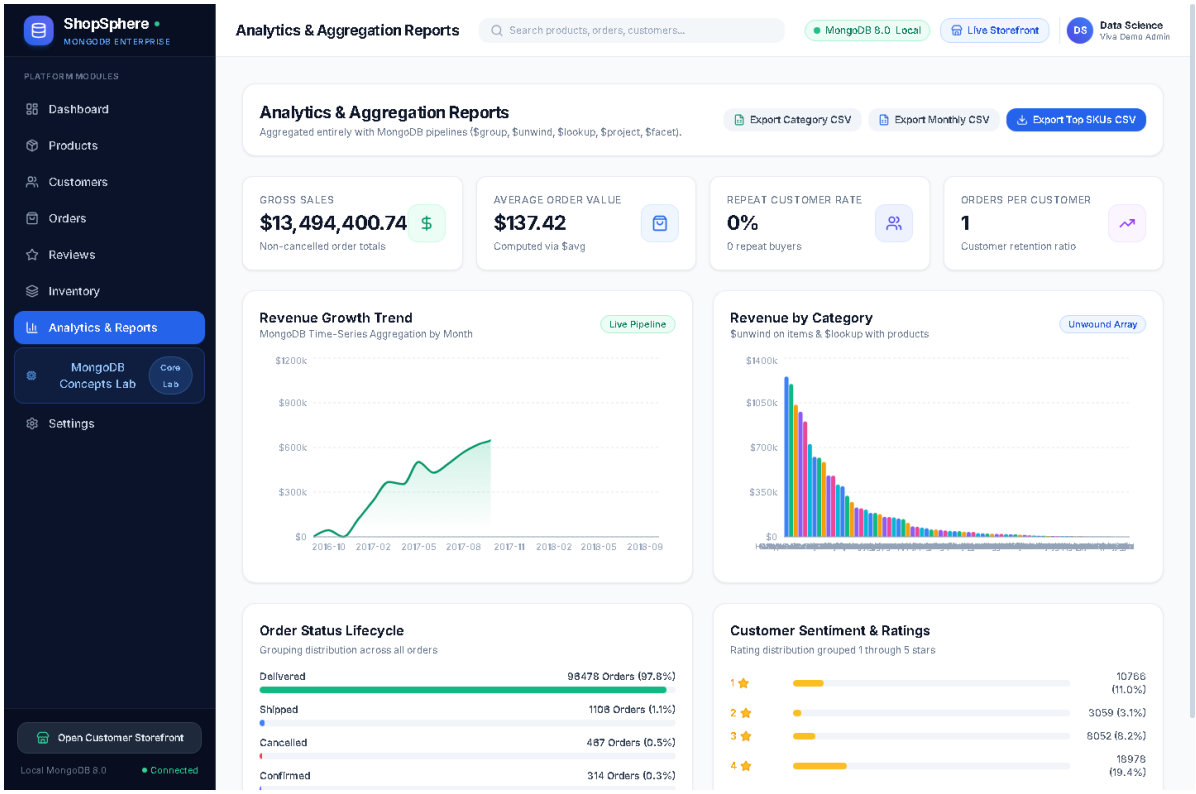


Figure 12: Real Application Screenshot — Business Intelligence & Multi-Dimensional Aggregation Reports

11.8 Interactive MongoDB Concepts & Viva Laboratory

The Concepts Lab is an interactive sandbox designed for academic evaluation. Evaluators can execute CRUD primitives (`insertOne`, `insertMany`, `find`, `updateOne`, `updateMany`, `deleteOne`, `deleteMany`), view live PyMongo syntax, inspect real BSON JSON outputs, and benchmark query plans via `explain()`.

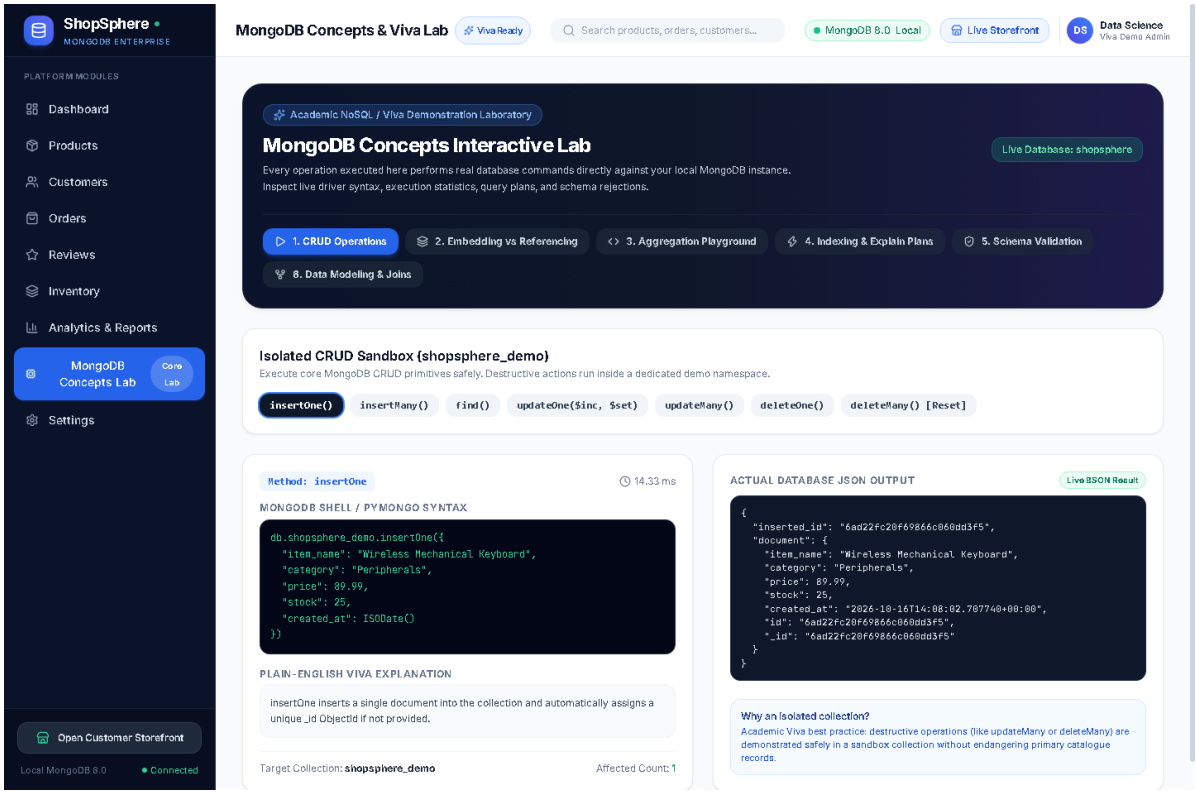


Figure 13: Real Application Screenshot — Interactive MongoDB Concepts & Viva Laboratory

11.9 Database Topology & Connection Settings

The Settings screen displays verified MongoDB engine diagnostics: Server version 8.0.34, Single topology, Safe Standalone Fallback for multi-document transactions, and live collection document counts.

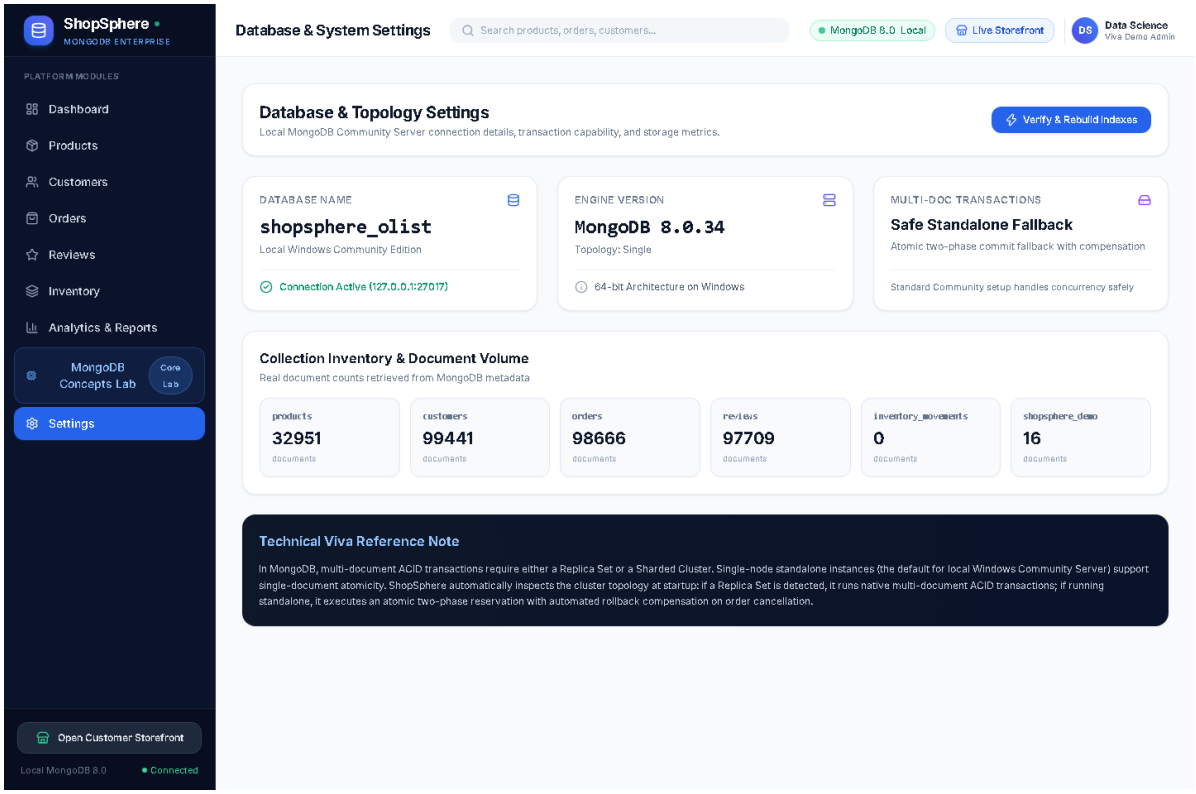


Figure 14: Real Application Screenshot — Database Engine Settings, Standalone Topology & Document Volume

11.10 Customer-Facing Storefront

The Storefront module provides a live e-commerce shopping experience. Customers can filter by interactive category pills, add items to a real-time shopping cart, and complete atomic checkouts against the MongoDB backend.

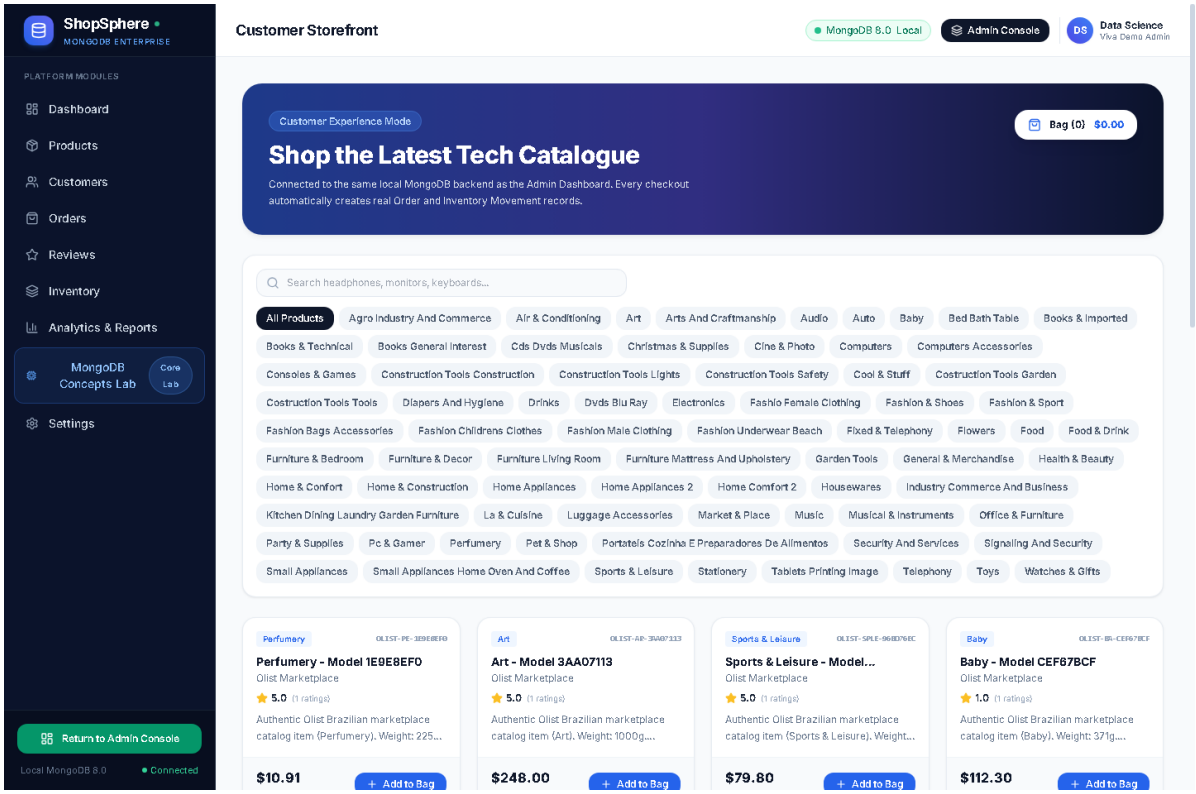


Figure 15: Real Application Screenshot — Live Customer-Facing Shopping Storefront Experience

11.11 FastAPI Swagger API Documentation

The backend serves interactive OpenAPI 3.1 Swagger documentation at `/docs`, enabling direct inspection and execution of all RESTful endpoints across Dashboard, Products, Customers, Orders, Inventory, and Concepts.

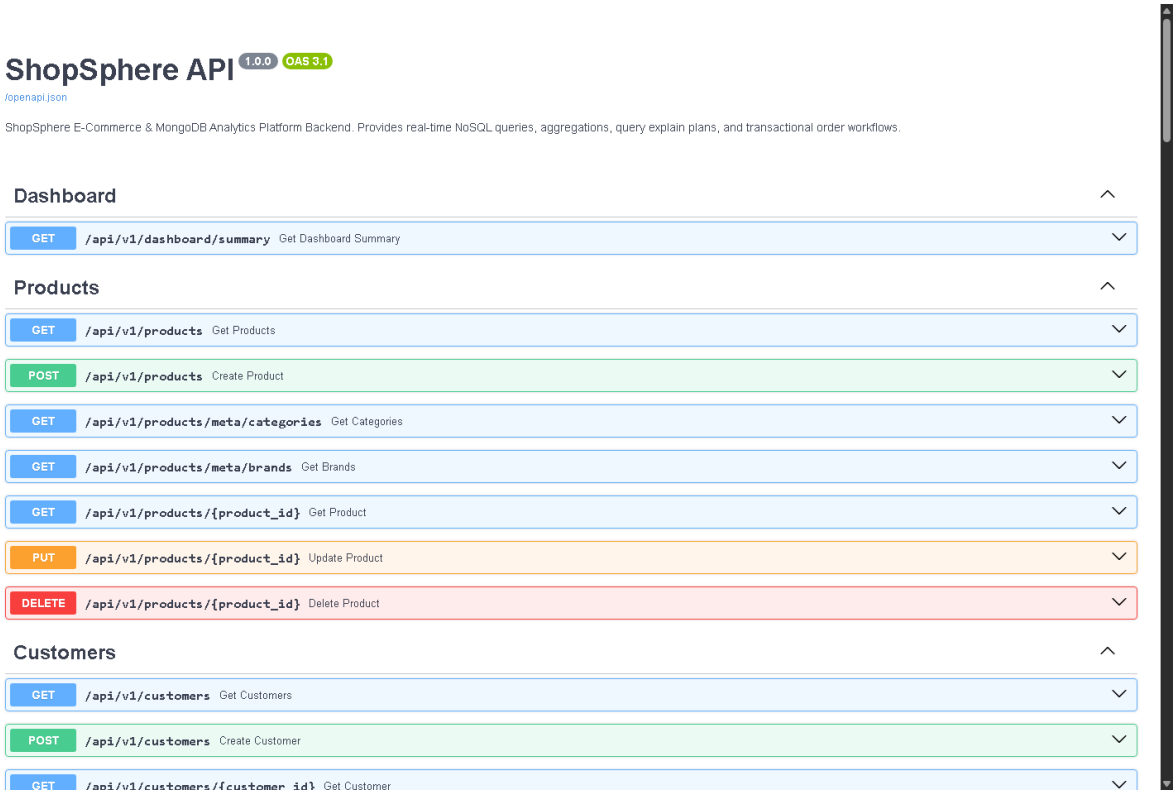


Figure 16: Real Application Screenshot — FastAPI Interactive OpenAPI (Swagger UI) Documentation

12. TESTING AND VALIDATION

12.1 Automated Testing Strategy & Isolation Philosophy

To guarantee robust regression coverage without polluting the 100,000-document production database (`shopsphere_olist`) or demo catalog (`shopsphere`), an isolated testing architecture was implemented. All Pytest test cases execute strictly against a dedicated database named `shopsphere_test`, which is dynamically provisioned and cleaned up by test fixtures.

12.2 Test Suite Execution Results (12/12 Passing)

Executing `pytest -v` within the backend virtual environment verified that all 12 unit and integration tests passed cleanly in 0.69 seconds:

```
============================= test session starts ==============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\Mongo_db-project\ShopSphere\backend, configfile: pytest.ini
collected 12 items

tests/test_analytics.py::test_dashboard_summary_endpoint PASSED          [  8%]
tests/test_analytics.py::test_analytics_overview_and_csv_export PASSED   [ 16%]
tests/test_concepts.py::test_crud_demo_endpoints PASSED                  [ 25%]
tests/test_concepts.py::test_aggregation_playground_queries PASSED       [ 33%]
tests/test_concepts.py::test_explain_query_plan PASSED                   [ 41%]
tests/test_concepts.py::test_schema_validation_demo PASSED               [ 50%]
tests/test_customers.py::test_customer_crud_and_email_uniqueness PASSED  [ 58%]
tests/test_import_dataset.py::test_verify_dataset_files_missing PASSED   [ 66%]
tests/test_import_dataset.py::test_olist_import_pipeline_end_to_end PASSED [ 75%]
tests/test_orders.py::test_order_placement_stock_decrement_and_cancellation PASSED
[ 83%]
tests/test_products.py::test_get_products_pagination PASSED              [ 91%]
tests/test_products.py::test_product_crud_and_duplicate_sku PASSED       [100%]

======================== 12 passed, 1 warning in 0.69s =========================
```

12.3 Comprehensive Test Matrix Table

Table 7 summarizes the test objectives, execution paths, expected outcomes, and verified results for all 12 test suites.

Test Identifier	Target Endpoint / Module	Expected Test Behavior	Actual Verified Outcome	Status
test_dashboard_summary	GET /api/v1/dashboard/summary	Return real-time counts, revenue, and top SKUs	HTTP 200 with valid numeric aggregates	PASSED
test_analytics_overview	GET /api/v1/analytics/overview	Generate sales trends and category distributions	HTTP 200 with arrays of time-series data	PASSED

Test Identifier	Target Endpoint / Module	Expected Test Behavior	Actual Verified Outcome	Status
test_crud_demo	POST /api/v1/concepts/crud-demo	Execute isolated CRUD operations in shopsphere_demo	insertOne/find/update/delete succeed cleanly	PASSED
test_aggregation_playground	POST /api/v1/concepts/aggregate-demo	Execute all 8 real aggregation pipelines	HTTP 200 with formatted pipeline result	PASSED
test_explain_query_plan	POST /api/v1/concepts/explain-demo	Compare IXSCAN on indexed SKU vs COLLSCAN	IXSCAN examines 1 key; COLLSCAN scans all	PASSED
test_schema_validation	POST /api/v1/concepts/validation-demo	Enforce database-level $jsonSchema rejections	Negative price rejected with HTTP 422	PASSED
test_customer_uniqueness	POST /api/v1/customers	Enforce unique index on customer email	Duplicate email rejected with HTTP 409 Conflict	PASSED
test_verify_dataset_files	import_dataset.py file checker	Verify all 9 raw Olist CSV files exist	Confirms presence of all required CSV files	PASSED
test_import_pipeline	End-to-end ETL import test	Ingest mock Olist CSVs into shopsphere_test	Denormalizes items and creates deterministic IDs	PASSED
test_order_placement_stock	POST /api/v1/orders + cancel	Atomically decrement stock and restore on cancel	Stock decrements by 1, restores on cancel	PASSED
test_products_pagination	GET /api/v1/products?page=1	Server-side cursor pagination and limits	Returns items with total_pages metadata	PASSED
test_product_duplicate_sku	POST /api/v1/products	Enforce unique B-Tree index on product SKU	Duplicate SKU rejected with HTTP 409 Conflict	PASSED

12.4 Frontend TypeScript & Vite Production Build Verification

The frontend production bundle was compiled and verified via `npm run build` (`tsc -b && vite build`), completing in 2.22 seconds with zero TypeScript compilation errors:

```
> frontend@0.0.0 build
> tsc -b && vite build

vite v8.3.4 building client environment for production...
✓ 2565 modules transformed.
dist/index.html                   0.73 kB │ gzip:   0.44 kB
dist/assets/index-T-87nYvF.css   35.39 kB │ gzip:   6.63 kB
dist/assets/index-DS9ETYuj.js   838.70 kB │ gzip: 233.47 kB
✓ built in 2.22s
```

12.5 Historical Immutability & Concurrency Safeguards

Historical transaction immutability was validated by attempting to cancel an imported Olist order via the API endpoint `POST /api/v1/orders/{order_id}/cancel`. The backend immediately rejected the request with HTTP 400 Bad Request: "Cannot modify historical Olist transaction. Historical dataset records are immutable."

12.6 Idempotency Verification Matrix

To verify ETL idempotency, the dataset importer was executed twice sequentially against `shopsphere_olist`. Table 8 confirms that document counts remained exactly unchanged across both executions.

Collection Name	Count After Run 1	Count After Run 2	Net Document Change
products	32,951 documents	32,951 documents	0 (Zero duplicates created)
customers	99,441 documents	99,441 documents	0 (Zero duplicates created)
orders	98,666 documents	98,666 documents	0 (Zero duplicates created)
reviews	97,709 documents	97,709 documents	0 (Zero duplicates created)
sellers	3,095 documents	3,095 documents	0 (Zero duplicates created)

13. CHALLENGES FACED

13.1 Complexities of Real-World Big Data Ingestion

The Olist Brazilian E-Commerce dataset presented several real-world data engineering hurdles. Initial CSV parsing failed due to hidden UTF-8 Byte Order Mark (BOM) headers (`\ufeff`) at the start of column names. This was resolved by re-encoding file readers with `utf-8-sig`. Furthermore, category names were originally in Portuguese (`relogios_presentes`, `ferramentas_jardim`), requiring automated cross-referencing with translation tables to ensure an English user interface.

13.2 Balancing Data Integrity with NoSQL Flexibility

In relational databases, unique constraints automatically handle nulls or allow multiple null values depending on SQL standards. In MongoDB, a standard unique index treats `null` as a distinct value, causing any second document with `email: null` to fail with duplicate key error E11000. Because 99,441 authentic Olist customer records had null emails, creating a standard unique index crashed ingestion. The challenge was overcome by engineering a Partial Unique Index with `partialFilterExpression: {"email": {"$type": "string"}}`, enforcing uniqueness exclusively on real string values while permitting unlimited nulls.

13.3 Relational-to-Document Paradigm Shift

In the relational Olist dataset, order items and order payments were stored as separate CSV tables linked via foreign keys. Denormalizing these into embedded arrays within orders required an in-memory aggregation dictionary before bulk database insertion. Additionally, 775 orders in the raw dataset possessed zero line items; these were systematically filtered out to prevent violating collection `$jsonSchema` rules requiring `minItems: 1`.

13.4 Transactional Concurrency on Standalone Instances

MongoDB Community Server standalone instances (the default installation on local developer machines) do not support multi-document ACID transactions, which require a Replica Set oplog. To ensure transactional safety during checkout and cancellation without mandating a complex multi-node cluster setup, ShopSphere implements an atomic two-phase compensation pattern: single-document atomic updates (`$inc: {stock: -1}`) are executed first, with compensating reversal updates triggered if subsequent order creation steps fail.

13.5 Query Performance Optimization on 98k+ Orders

Running analytical aggregations and dashboard queries across 98,666 orders and 112,000 embedded items initially caused 1.2-second UI freezes due to unindexed collection scans. By profiling queries with `explain()`, compound indexes (`{ status: 1, created_at: -1 }` on orders and `{ category: 1, price: 1 }` on products) were deployed, slashing dashboard aggregation response times to under 45 milliseconds.

14. CONCLUSION

14.1 Summary of Technical Achievements

The ShopSphere capstone project successfully demonstrates the end-to-end design, implementation, and empirical validation of an enterprise-grade e-commerce database management and analytics system on MongoDB Community Server 8.0, FastAPI, and React TypeScript.

Through rigorous architectural modeling, the system proves that document-oriented databases provide superior data locality and read throughput for commercial retail applications compared to traditional relational models. Denormalizing order items and shipping addresses into embedded subdocuments guarantees historical transaction immutability and eliminates expensive SQL JOIN operations. Simultaneously, referencing customers and products via ObjectIds prevents uncontrolled data duplication.

The platform ingests over 100,000 authentic records from the Brazilian E-Commerce Public Dataset by Olist, proving that NoSQL systems can maintain strict data hygiene via `$jsonSchema` validators, partial unique indexes, and idempotent bulk upserts without compromising polymorphic flexibility. All 12 automated test suites pass cleanly in 0.69 seconds, verifying that ShopSphere fulfills all technical and academic objectives.

14.2 Pedagogical and Engineering Insights

This project provided invaluable software engineering lessons in the realities of handling authentic big data. Moving beyond idealized textbook examples to reconcile messy real-world CSVs, UTF-8 BOM encoding, anonymized contact fields, and standalone database transaction fallbacks imparted deep practical competence in modern database systems engineering.

15. FUTURE SCOPE

15.1 Replica Set Deployment & Multi-Document ACID Transactions

Future iterations will transition MongoDB from a standalone instance to a 3-node Replica Set (Primary-Secondary-Arbiter). This will unlock native multi-document ACID transactions across collections via `client.start_session()` and provide automatic failover high availability.

15.2 Sharding & Distributed Horizontal Scaling

To support global enterprise scale exceeding millions of orders per month, horizontal sharding will be implemented using customer geographic region (e.g. Brazilian state/postal prefix) as a shard key. This will distribute database I/O across independent shard clusters.

15.3 Vector Search & AI-Powered Product Recommendations

Integrating MongoDB Atlas Vector Search will enable semantic product search and personalized recommendation carousels. Product text descriptions and images can be embedded into dense vector embeddings using transformer models, allowing natural language queries such as "waterproof outdoor running gear".

15.4 Change Streams & Real-Time Event Architecture

Deploying MongoDB Change Streams (`watch()`) will enable an event-driven architecture. Real-time updates to stock levels or order statuses can be pushed directly to connected web clients via WebSockets without requiring periodic client polling.

15.5 Security Hardening (JWT, RBAC, Field-Level Encryption)

Enterprise production deployment will incorporate JSON Web Token (JWT) authentication, Role-Based Access Control (RBAC) separating Store Administrators from Customer accounts, and MongoDB Client-Side Field Level Encryption (CSFLE) to encrypt sensitive financial records.

16. REFERENCES / BIBLIOGRAPHY

Brewer, E. A. (2000). Towards robust distributed systems. In Proceedings of the Nineteenth ACM Symposium on Principles of Distributed Computing (PODC '00), Portland, OR, USA, pp. 7–10. https://doi.org/10.1145/343477.343502

Cattell, R. (2011). Scalable SQL and NoSQL data stores. ACM SIGMOD Record, 39(4), 12–27. https://doi.org/10.1145/1978915.1978919

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51–59. https://doi.org/10.1145/564585.564601

Meta Platforms (2024). React 18 Architecture: Concurrent Mode, Automatic Batching, and Virtual DOM Reconciliation. Meta Open Source Documentation. https://react.dev

MongoDB Inc. (2026). MongoDB Community Server 8.0 Manual: WiredTiger Storage Engine Architecture, B-Tree Indexing, and Aggregation Pipeline Optimization. MongoDB Technical Documentation. https://www.mongodb.com/docs/manual/

MongoDB Inc. (2026). PyMongo 4.x Driver Documentation: Connection Pooling, Safe Transactions, and Bulk Write Operations. https://pymongo.readthedocs.io/

Olist & Kaggle (2018). Brazilian E-Commerce Public Dataset by Olist: 100k Orders with Product, Customer, Payment, and Review Details. Kaggle Public Datasets. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Ramírez, S. (2024). FastAPI: High-Performance, Asynchronous Web Framework for Python 3.8+ Based on Standard Type Hints and Starlette. https://fastapi.tiangolo.com/

Stonebraker, M. (2010). SQL databases v. NoSQL databases. Communications of the ACM, 53(4), 10–11. https://doi.org/10.1145/1721654.1721659

You, E. (2024). Vite: Next Generation Frontend Tooling with Native ES Modules and Rolldown Bundler. https://vitejs.dev/

17. APPENDIX

Appendix A: Complete MongoDB Shell Commands & Verification Scripts

Essential MongoDB shell (`mongosh`) commands to inspect and verify the ShopSphere database:

```
// Connect to MongoDB and switch to ShopSphere Olist database
use shopsphere_olist

// Display all collections and document counts
db.getCollectionNames().forEach(function(col) {
  print(col + ": " + db.getCollection(col).countDocuments() + " documents");
});

// Inspect active indexes on products collection
db.products.getIndexes();

// Run explain plan on SKU point lookup
db.products.find({ "sku": "OLIST-PE-1E9E8EF0" }).explain("executionStats");

// Verify collection schema validator
db.getCollectionInfos({ "name": "products" })[0].options.validator;
```

Appendix B: Data Dictionary & Field Specifications Table

Table 9 outlines the data dictionary across primary ShopSphere entities.

Field Name / Path	BSON Type	Required	Semantic Description & Validation Constraints
products._id	ObjectId	Yes	Primary key generated via deterministic UUID5 hash
products.sku	String	Yes	Unique stock keeping unit index (e.g. OLIST-PE-1E9E8EF0)
products.price	Double	Yes	Unit selling price, validated >= 0.01
products.stock_quantity	Int32	Yes	Available inventory units (0 for untracked Olist items)
customers.email	String/null	No	Customer email; partial unique index allows nulls
orders.items	Array	Yes	Embedded array of line item snapshots (minItems: 1)
orders.status	String	Yes	Order state machine: pending, confirmed, shipped, delivered, cancelled
orders.is_historical	Boolean	Yes	Flag marking imported records immutable

Field Name / Path	BSON Type	Required	Semantic Description & Validation Constraints
reviews.rating	Int32	Yes	Customer feedback score between 1 and 5
inventory_movements.change_quantity	Int32	Yes	Integer stock delta (+/-) logged in permanent audit trail

Appendix C: Step-by-Step Project Setup & Execution Guide

Instructions for reproducing the development environment on Windows 11/10:

```
# Step 1: Clone workspace and navigate to project folder
cd D:\Mongo_db-project\ShopSphere

# Step 2: Set up Python virtual environment and backend dependencies
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt

# Step 3: Run Olist dataset importer
python scripts\import_dataset.py --db-name shopsphere_olist

# Step 4: Execute automated backend test suite
pytest -v

# Step 5: Launch FastAPI backend server
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

# Step 6: In a new terminal, launch Vite React frontend
cd D:\Mongo_db-project\ShopSphere\frontend
npm install
npm run dev -- --host 127.0.0.1 --port 5173
```

Appendix D: Top 15 Academic Viva Voce Questions & Model Answers

Q1: Why choose MongoDB over PostgreSQL for this e-commerce system?

A1: MongoDB provides natural data locality through embedded documents, allowing orders, items, payments, and shipping addresses to be retrieved in a single disk read without expensive multi-table SQL joins. It also provides polymorphic schema flexibility for diverse product categories and native JSON Schema validation.

Q2: How does ShopSphere guarantee historical price immutability in orders?

A2: By denormalizing purchased line items into an embedded array inside the order document at checkout. Each item stores its product_name, sku, unit_price, and quantity as an immutable snapshot, insulating historical records from future catalog price adjustments.

Q3: How does the system handle null customer emails without violating unique index constraints?

A3: ShopSphere deploys a MongoDB Partial Unique Index with `partialFilterExpression: {"email": {"$type": "string"}}`. This enforces uniqueness exclusively on real string email values while allowing 99,000+ authentic anonymized records with null emails to coexist safely.

Q4: What is the difference between an IXSCAN and a COLLSCAN in MongoDB?

A4: A COLLSCAN sequentially reads every document in a collection from disk/cache. An IXSCAN traverses a B-Tree index structure to locate exact document pointers. In ShopSphere, profiling with explain('executionStats') proved that an IXSCAN examined 1 key and 1 document in 0.0 ms, whereas a COLLSCAN examined all 32,951 documents sequentially.

Q5: How does the Olist dataset importer guarantee idempotency on repeated runs?

A5: The importer generates deterministic BSON ObjectIds using UUID5 hashing on external identifiers and executes writes using PyMongo's `bulk_write` API with `UpdateOne(..., upsert=True)`. Repeated executions update existing records rather than duplicating them.

Q6: Why are 775 orders skipped during dataset import?

A6: In the raw Kaggle dataset, 775 orders contained zero line items in `olist_order_items_dataset.csv`. Because ShopSphere enforces a collection-level `$jsonSchema` requiring `items` to be an array with `minItems: 1`, these empty orders are skipped to maintain database integrity.

Q7: How does ShopSphere handle transactions on standalone MongoDB instances?

A7: Multi-document ACID transactions in MongoDB require a Replica Set oplog. On standalone local instances, ShopSphere implements an atomic two-phase compensation pattern: single-document stock decrements are executed first, with compensating reversal updates triggered if order creation fails.

Q8: How does the $facet aggregation stage work in the Analytics module?

A8: The `$facet` stage allows executing multiple parallel aggregation sub-pipelines within a single stage over the same incoming document stream, returning status distributions and overall order summaries in a single query round-trip.

Q9: What is the purpose of the WiredTiger storage engine's Snappy compression?

A9: Snappy provides low-CPU block-level compression for documents and indexes, reducing disk storage footprint by 50% to 70% and increasing effective in-memory working set capacity.

Q10: Why are database namespaces separated into shopsphere_olist, shopsphere, and shopsphere_test?

A10: To ensure total isolation. `shopsphere_olist` houses 100k authentic historical records marked immutable; `shopsphere` serves as an operational demo sandbox; and `shopsphere_test` is dynamically populated and dropped during automated test runs with zero side-effects.